

DUMPSTER FIRE

Tomo 2

Sistemas técnicos y presentación

Jugador, daño, cámara, feedback, UI, audio, infraestructura y herramientas

Documentación técnica interna · uso del equipo de desarrollo

Estado: alpha · 23-09-2026

Base de código: rama **main**, commit **53ef237** · Unity 6000.3.13f1 · URP 17.3

Contenido

Las referencias a los documentos 01–07 (GDD, Overheat, Spawning, Game Loop, Progresión, QA, Deltas) remiten a la documentación alpha del 22-09-2026, que no forma parte de esta carpeta. Las rutas `archivo:línea` son relativas a `Assets/Scripts/` salvo que se indique otra cosa. Cuando un valor serializado en escena o prefab difiere del default del script, manda el de la escena.

PARTE 1 · JUGADOR

1	Movimiento	4
2	Animación del jugador	9
3	Rear threat	14

PARTE 2 · COMBATE Y PRESENTACIÓN

4	Daño	20
5	Feedback de combate	24
6	Cámara y aim	30
7	UI y HUD	33
8	Audio	38

PARTE 3 · INFRAESTRUCTURA

9	Escenas y navegación	42
10	Settings y accesibilidad	45
11	Dormancia, pools y rendimiento	48
12	Herramientas de editor y tests	52

PARTE 1

Jugador

1 Movimiento

Movimiento del jugador sobre `Rigidbody` dinámico, relativo a cámara, con salto, saltos aéreos, dash por cargas, agacharse y slide. No hay máquina de estados explícita: el estado son flags (`_isGrounded`, `_isCrouching`, `_isSliding`, `_isDashing`) más timers (stun, knockback, post-dash, slow, aim-facing). El input se lee en `Update` y la física corre en `FixedUpdate` (paso fijo 0,02 s, `ProjectSettings/TimeManager.asset`). Las stats de movimiento salen de `PlayerStats`.

Scripts

Script	Rol
<code>Assets/Scripts/Player/PlayerMovement.cs</code>	Input, aceleración, fricción, tope de velocidad, salto, dash, crouch/slide, efectos externos (knockback, pull, stun, slow, weapon dash)
<code>Assets/Scripts/Player/PlayerStats/PlayerStats.cs</code>	Fuente de <code>MovementSpeed</code> , <code>JumpHeight</code> , <code>AirJumps</code> , <code>DashCharges</code> , <code>DashSpeed</code>
<code>Assets/Scripts/ThirdPersonCamera.cs</code>	Orbit de cámara con mouse. <code>PlayerMovement</code> usa su <code>forward/right</code> aplanados
<code>Assets/Scripts/Powerups/TemporaryPowerupController.cs</code>	Buff <code>ExtraSpeed</code> : multiplica velocidad y salto, suma dashes y saltos aéreos
<code>Assets/Prefabs/player.prefab</code>	<code>Rigidbody</code> , collider y valores serializados de <code>PlayerMovement</code>

`PlayerMovement` tiene `[DefaultExecutionOrder(-100)]` y exige `Rigidbody` y `PlayerStats` (`PlayerMovement.cs:5-7`). Expone `PlayerTransform` estático (singleton, `:10-12`) y eventos para animación y HUD: `OnJump`, `OnAirJump`, `OnLanded`, `OnCrouchStarted/Ended`, `OnSlideStarted/Ended`, `OnDashStarted/Ended`, `OnDashChargesChanged`, `OnStunned` (`:14-24`).

Física base

Parámetro	Valor	Fuente
Gravedad	-18	<code>ProjectSettings/DynamicsManager.asset:7</code>
Fixed timestep	0,02 s	<code>ProjectSettings/TimeManager.asset</code>
Masa <code>Rigidbody</code>	1, sin damping	<code>player.prefab:764-765</code>
Interpolación	Prefab 0, forzada a <code>Interpolate</code> en <code>Awake</code>	<code>PlayerMovement.cs:274</code>
Rotación física	Congelada en <code>Awake</code>	<code>PlayerMovement.cs:275</code>
Collider de movimiento	Capsule r 0,5 · h 2	<code>player.prefab:529-551</code>
Escala del jugador en escena	1,2 (cápsula efectiva h 2,4)	override en <code>GameplayScene.unity</code>
Material de fricción	Runtime, fricción 0, combine Minimum	<code>PlayerMovement.cs:279-293</code>

El prefab también tiene un `CharacterController` deshabilitado en la raíz (`player.prefab:552-576`) y un hurtbox trigger (`ContactHurtbox`, r 0,62) con su propio `Rigidbody` kinemático.

Valores serializados

Prefab = `Assets/Prefabs/player.prefab:589-611`. La escena solo sobrescribe `_cameraTransform` y `_baseMoveAcceleration` (75, igual al prefab).

Campo	Default código	Prefab / escena	Uso
<code>_baseMoveAcceleration</code>	38	75	Aceleración por input (m/s ²)
<code>_baseFriction</code>	9	10	Desaceleración constante (m/s ²)
<code>_airFrictionMultiplier</code>	0,2	0,4	Fricción en el aire
<code>_crouchAccelerationMultiplier</code>	0,2	0,2	Aceleración agachado
<code>_slideFrictionMultiplier</code>	0,1	0,4	Fricción en slide (solo en suelo)
<code>_dashDuration</code>	0,25	0,25	Duración del dash (s)
<code>_groundedDashRegenTime</code>	2	2	Recarga de 1 carga en suelo (s)
<code>_airborneDashRegenTime</code>	4	4	Recarga de 1 carga en aire (s)
<code>_slideStartSpeedMultiplier</code>	1,5	1,5	Umbral de slide = MoveSpeed × 1,5
<code>_minSlideSpeed</code>	2	2	Debajo de esto el slide termina
<code>_postDashFrictionMultiplier</code>	0,15	0,3	Fricción tras el dash
<code>_postDashFrictionDuration</code>	0,18	0,18	Ventana post-dash (s)
<code>_knockbackWindow</code>	0,3	0,3	Ventana sin tope tras empuje (s)
<code>_knockbackFrictionMultiplier</code>	0,1	0,1	Fricción en la ventana de empuje
<code>_groundCheckExtraDistance</code>	0,08	0,08	Margen del raycast de suelo
<code>_groundMask</code>	Everything	Everything	Capas que cuentan como suelo
<code>_facingSmoothTime</code>	0,12	0,12	Suavizado del giro (s)
<code>_strafeTurnAngle</code>	25	25	Giro extra al strafear (°)
<code>_aimFacingRotationSpeed</code>	720	720	Giro forzado por arma (°/s)

Stats que consume

Assets en `Assets/ScriptableObjects/PlayerSO/Stats/`. Detalle en `41-Stats-del-jugador.md`.

Stat	Base	Level-up	Uso en código
<code>MovementSpeed</code>	6,8	Sí, 0,04	Tope de velocidad planar y umbral de slide (<code>PlayerMovement.cs:441,470,678</code>)
<code>JumpHeight</code>	3	Sí, 0,0025	Altura de salto (:578)
<code>AirJumps</code>	0	No	Saltos aéreos, se recargan al aterrizar (:763)
<code>DashCharges</code>	1	No	Cargas máximas (:686,727)
<code>DashSpeed</code>	10	Sí, 0,025	Velocidad sumada por el dash (:697)

Derivados con los valores actuales: velocidad de salto $\sqrt{2 \cdot 18 \cdot 3} \approx 10,4$ m/s, subida $\approx 0,58$ s. Umbral de slide $6,8 \times 1,5 = 10,2$ m/s. Buff ExtraSpeed: `MovementSpeed` ×2–4 y `JumpHeight` ×2–3 durante 15 s; dashes y saltos aéreos +2× lo que ya tenga el jugador (`TemporaryPowerupController.cs:112-124`). Con `AirJumps` base 0 el buff no da saltos aéreos si no hay pasivo.

Flujo por frame

1. Update (`PlayerMovement.cs:312-333`): si hay pausa de UI o `timeScale` 0, sale. Lee input. Si no está aturdido, dispara `TryJump`, `TryStartCrouchOrSlide`, `StopCrouchOrSlide`, `TryDash`. Los flags de "pressed" se consumen en

el mismo frame: no hay buffer.

2. `FixedUpdate` (:336-361): `UpdateGroundedState` → retiene stun con momentum → si dashea, solo corre el timer; si no, `HandleMovement` → `ApplyPlanarSpeedCap` → `HandleFriction`. Después, ticks de post-dash, knockback, stun, slow, aim-facing y recarga de dash.

Movimiento en suelo y aire

1. Dirección: `forward` y `right` de la cámara aplanados en XZ, combinados con WASD y normalizados (:386-389).
2. Giro: el cuerpo mira hacia la cámara con +25° de strafe con A/D. En slide conserva el rumbo del slide. Si un arma pidió `RequestAimFacing`, gira a 720°/s hacia ese punto (:412-432).
3. Aceleración: $75 \times \text{lerp}(1, 0, 35, v / v_{\text{max}})$, $\times 0,2$ agachado. No acelera durante el slide (:434-445).
4. Tope: si $v > \text{MovementSpeed} \times \text{slow}$, recorta la velocidad planar. No aplica en slide, post-dash ni knockback (:466-477).
5. Fricción: aceleración constante opuesta a la velocidad planar. Base 10; $\times 0,4$ en aire; $\times 0,4$ en slide en suelo; $\times 0,3$ post-dash; $\times 0,1$ en knockback (:449-462). Los multiplicadores se encadenan.
6. Suelo: raycast hacia abajo desde el centro del collider, largo `extents.y + 0,08`, triggers ignorados (:789-801). Al aterrizar recarga `AirJumps` y emite `OnLanded`.

Salto

1. En suelo, salto normal. En aire, consume un salto aéreo si queda (:560-573).
2. Pone en 0 la velocidad vertical si es negativa y aplica impulso $\sqrt{2 \cdot g \cdot \text{JumpHeight}}$. El salto aéreo tiene la misma altura (:576-591).
3. Cancela el slide. No hay coyote time ni jump buffer: al salir de un borde el salto ya cuenta como aéreo.

Dash

1. Requiere `DashCharges` ≥ 1 , una carga disponible e input de dirección. Sin WASD no dashea (:682-690).
2. Corta slide y crouch. Suma `DashSpeed` (10 m/s) a la velocidad planar actual, en la dirección de input (:697-703). Durante 0,25 s no hay aceleración, fricción ni tope; la gravedad sigue.
3. Al terminar: ventana post-dash de 0,18 s con fricción $\times 0,3$ y sin tope. Si Ctrl sigue apretado, intenta slide (:712-722).
4. Recarga: una carga cada 2 s en suelo o 4 s en aire. El timer se reinicia en cada dash (:706,725-753).
5. `ApplyWeaponDash` (armas) usa el mismo estado con velocidad absoluta y no consume cargas (:242-264).

Crouch y slide

1. Ctrl presionado: si está en suelo y $v \geq 10,2$ m/s, slide. Si no, crouch (:594-611). Crouch solo baja la aceleración a $\times 0,2$; no cambia collider ni altura.
2. El slide fija su dirección a la velocidad actual, no acelera, no tiene tope y usa fricción $\times 0,4$ (:639-654).
3. Termina al soltar Ctrl, al saltar o al bajar de 2 m/s en suelo (entonces pasa a crouch si Ctrl sigue apretado) (:773-783).
4. Aterrizar con Ctrl apretado arranca slide sin mirar la velocidad (:766-770).
5. Con correr normal no se llega al umbral (tope $6,8 < 10,2$): en la práctica el slide sale de dash, knockback, caída o aterrizaje con Ctrl.

Efectos externos

API	Qué hace	Línea
<code>ApplyKnockback(from, force)</code>	Impulso horizontal <code>force × masa</code> y ventana de 0,3 s sin tope y fricción <code>×0,1</code>	:151-164
<code>ApplyPull(toward, accel)</code>	Aceleración continua hacia un punto. Reusa la ventana de knockback; se llama cada frame	:171-184
<code>ApplyStun(s)</code>	Bloquea input de movimiento, salto y dash. Refresca, no apila. La fricción sigue	:187-196
<code>ApplyMomentumPreservingStun</code>	Congela la velocidad planar, corta dash y slide y la restituye al final con ventana post-dash	:198-226, 500-527
<code>ApplySlow(mult, s)</code>	Multiplca el tope (0,05–1). Conserva el más lento y la duración más larga	:232-240
<code>RefreshPassiveResources()</code>	Rellena saltos aéreos y cargas de dash al cambiar pasivos o buffs	:135-145

Input

`PlayerMovement` no usa el Input System por acciones. Lee `Keyboard.current` directo (:374-398, 827-841):

Acción en código	Tecla
Mover	W A S D
Saltar	Space
Dash	Left/Right Shift
Crouch / slide	Left/Right Ctrl (mantener)
Mirar	Delta de <code>Mouse.current</code> (<code>ThirdPersonCamera.cs:260-268</code>)

`Assets/InputSystem_Actions.inputactions` es el asset de plantilla de Unity. Está registrado como acciones del proyecto (`ProjectSettings/EditorBuildSettings.asset:28`), pero ningún script de gameplay lo lee. El `InputSystemUIInputModule` de la escena usa otro asset (guid `ca9f5fa9...`, `GameplayScene.unity:23533`). Esquemas: Keyboard&Mouse, Gamepad, Touch, Joystick, XR.

Mapa `Player`:

Acción	Teclado / mouse	Gamepad	Usada por código
Move (Vector2)	WASD, flechas	Left stick	No (WASD directo; flechas no)
Look (Vector2)	Pointer delta	Right stick	No (mouse directo)
Attack	Mouse izq., Enter	West	No
Interact (Hold)	E	North	No
Crouch	C	East	No (el código usa Ctrl)
Jump	Space	South	No (Space directo)
Previous / Next	1 / 2	D-pad izq. / der.	No
Sprint	Left Shift	Left stick press	No (Shift es dash)

Mapa UI: Navigate, Submit, Cancel, Point, Click, RightClick, MiddleClick, ScrollWheel, TrackedDevicePosition/Orientation, con los bindings de plantilla. Solo `TitleScreenScreenStack.cs:50` referencia una acción por `InputActionReference` (cancel).

Cámara

Valores de escena (`GameplayScene.unity:5410-5426`): pivote a 1,5 m, hombro +1, distancia 3,5, sensibilidad 0,12 / 0,12, pitch -70° a 70° , cursor bloqueado en Play. La cámara calcula el orbit en `Update` y lo escribe en `LateUpdate`, sumando shake y FOV kick (`ThirdPersonCamera.cs:289-296`). `PlayerMovement` lee el transform de la cámara, así que toma la orientación del frame anterior, con el shake incluido.

Deuda / puntos abiertos

- Input hardcodeado a teclado y mouse (`PlayerMovement.cs:374-398`, `ThirdPersonCamera.cs:260`). Sin gamepad, sin rebinding, y el asset de acciones contradice el código: Crouch = C y Sprint = Shift en el asset, Ctrl y dash en el código.
- Debug.Log sin guardas en cada intento de slide, entrada y salida (`PlayerMovement.cs:598,602,651,661`). Llenan la consola y se compilan en release.
- Sin coyote time ni jump buffer (`:323,560-573`). Un salto apretado justo antes de aterrizar se pierde, y uno apretado al salir de un borde gasta un salto aéreo (con `AirJumps 0`, no salta).
- La fricción es una desaceleración constante sin corte en cero (`:449-462`). Con $10 \text{ m/s}^2 \times 0,02 \text{ s}$ puede invertir el signo de velocidades menores a 0,2 m/s y dejar un temblor en reposo.
- El umbral de slide (`MoveSpeed \times 1,5 = 10,2`) supera el tope de carrera (6,8). El slide no se alcanza corriendo, solo con dash, empuje o aterrizaje con Ctrl (`:676-679,766-770`). Hay que confirmar si es intencional.
- Crouch no cambia collider ni altura: solo baja la aceleración (`:437`). No tiene efecto defensivo ni de navegación.
- La dirección de movimiento usa el transform de la cámara con el shake aplicado (`ThirdPersonCamera.cs:294-295`), hasta $\pm 5^\circ$ de rotación. Debería usar la rotación de gameplay.
- `_groundMask = Everything` (`player.prefab:596-598`): enemigos y props no trigger cuentan como suelo y recargan saltos aéreos.
- Los incrementos por nivel de `JumpHeight` (0,0025) y `DashSpeed` (0,025) son despreciables frente a sus bases (3 y 10). Ocupan slots de la ruleta sin efecto visible.
- Restos en el prefab: `CharacterController` deshabilitado (`player.prefab:552-576`). El override de escena de `_baseMoveAcceleration = 75` repite el valor del prefab.

2 Animación del jugador

La animación del jugador es solo presentación: no mueve al personaje ni decide el disparo. Un `AnimatorController` generado se evalúa a mano desde un `PlayableGraph`. Encima se aplican ajustes procedurales acotados: dash direccional, braceo con arma, aim de pecho y brazos, y retroceso. Las armas piden la pose antes de leer la boca del cañón, así el muzzle sale del esqueleto ya apuntado. Los enemigos usan otra cosa, un `Animator` clásico de un solo estado armado por un builder de editor.

Scripts

Script	Rol
<code>Assets/Scripts/Player/Animation/PlayerAnimationDriver.cs</code>	Grafo manual, selección de estados, aim/recoil procedural
<code>Assets/Scripts/Player/Animation/PlayerWearableSurfaceFollower.cs</code>	Pega los sockets de armas rígidas a la piel deformada
<code>Assets/Scripts/Editor/PlaceholderPlayerAnimationBuilder.cs</code>	Importa el FBX, genera la máscara, el controller y los sockets del prefab
<code>Assets/Scripts/Editor/PlayerWearableSurfaceBuilder.cs</code>	Calibra las anclas de superficie a partir de los skin weights
<code>Assets/Scripts/Editor/EnemySkinnedAnimBuilder.cs</code>	Reemplaza el visual estático de los enemigos por el FBX skineado y crea sus controllers
<code>Assets/Scripts/Enemy/EnemyAnimPhaseOffset.cs</code>	Desfasa el clip del Slime al activarse

Assets: `Assets/Art/Player/PlaceholderAnimation/PlaceholderPlayer.fbx`, `PlaceholderPlayer.controller`, `UpperBody.mask`. El driver vive en `Assets/Prefabs/player.prefab`, y la escena no le hace overrides.

Grafo (Playables)

`EnsureGraph` (`PlayerAnimationDriver.cs:267`):

1. Guarda `applyRootMotion` y `cullingMode`, pone root motion en off y `AlwaysAnimate`.
2. Le saca el `runtimeAnimatorController` al `Animator` (:277) para que el pase normal de Unity no pise la pose.
3. `PlayableGraph` en `DirectorUpdateMode.Manual`, un único `AnimatorControllerPlayable` sobre el controller, y un `AnimationPlayableOutput` hacia el `Animator` (:278-282).
4. Arranca en `Locomotion` / `Aim` / `Empty`, con el peso de `Upper Body` en 0.

No hay `AnimationMixerPlayable` ni clips sueltos en el grafo. Las capas y la máscara salen del propio controller. `DestroyGraph` (:294) devuelve el controller y los flags originales al deshabilitar el componente o al resetear la presentación.

Capas del controller

Lo genera `BuildController` (`PlaceholderPlayerAnimationBuilder.cs:249`). Todas las capas son `Override`: no hay capas aditivas.

Capa (índice)	Máscara	Peso por defecto	Estados
Base Layer (0)	ninguna	1	Locomotion (2D Freeform Directional: Idle, MoveForward/Backward/Left/Right), Crouch (1D por <code>Speed</code> : CrouchIdle/CrouchMove), Jump, AirJump, Fall, Land, Slide, Dash, DashBackward, Stun

Capa (índice)	Máscara	Peso por defecto	Estados
Upper Body (1)	UpperBody .mask	1 en el asset, 0 al crear el grafo	Aim, Fire, Slash, Flame, Hit
Full Body (2)	ninguna	0	Empty (sin motion), Death

- La máscara activa `spine.003` y todos sus hijos (`PlaceholderPlayerAnimationBuilder.cs:200-206`). Desde la espalda baja hacia abajo manda la locomoción.
- Parámetros: `MoveX`, `MoveY`, `Speed`, `LocomotionRate`. `Locomotion` y `Crouch` usan `LocomotionRate` como multiplicador de velocidad.
- El controller no tiene transiciones útiles. Todos los cambios los hace el driver con `CrossFadeInFixedTime (:429-434)`.

Clips (FBX, PlaceholderPlayer.fbx.meta)

Clip	Frames	Loop	Clip	Frames	Loop
Idle	0–120	sí	Slide	0–30	sí
MoveForward/Backward/Left/Right	0–22	sí	Dash	0–12	no
CrouchIdle	0–120	sí	DashBackward	0–9	no
CrouchMove	0–32	sí	Stun	0–40	sí
Jump	0–12	no	Aim	0–120	sí
AirJump	0–14	no	Fire	0–8	no
Fall	0–30	sí	Slash	0–16	no
Land	0–10	no	Flame	0–30	sí
Hit	0–10	no	Death	0–36	no

Import Generic, compresión Off, root bloqueado en XZ, Y y rotación (`PlaceholderPlayerAnimationBuilder.cs:37-45`, `:128-146`). El FBX sale de `Tools/Blender/generate_player_animations.py`.

Menús: `Tools/ScrapWaves/Build Placeholder Player Animation`, `Refresh Player Animation Controller`, `Refit Animated Wearable Sockets`, `Render Placeholder Player Animation`.

Orden de evaluación

1. `WeaponManager.Update` limpia el override manual y llama `EvaluatePoseForWeapons(dt, CurrentAimSolution.TargetPoint)` (`WeaponManager.cs:69-73`). Después resuelve el aim de nuevo con el muzzle actualizado y recién ahí tickea las armas.
2. `EvaluatePoseForWeapons` (`PlayerAnimationDriver.cs:328`) corre una sola vez por frame (`_lastEvaluatedFrame`): locomoción, upper body, `_graph.Evaluate(dt)`, y después dash direccional, braceo con arma, aim, recuperación y cacheo de pose, recoil y wearables.
3. Si nadie evaluó en el frame (pausa, `GameManager` fuera de `Playing`), `LateUpdate (:357)` evalúa igual, contra `CurrentAimSolution` o el reticle.
4. Muerto: el grafo avanza con `unscaledDeltaTime (:332)` para que `Death` corra durante la pausa de game over. No se aplica aim ni recoil.

5. `PlayerWearableSurfaceFollower.EvaluateAttachments` se ejecuta al final (:353), antes de que cualquier arma lea sockets.

`[DefaultExecutionOrder(100)]` en el driver.

Lectura del movimiento

`UpdateLocomotion` (:368) lee `PlayerMovement.CurrentVelocity` en espacio local:

- `MoveX/MoveY` = velocidad local / `_referenceMoveSpeed`, con clamp a 1 y suavizado exponencial (`_velocityBlendTime`).
- `LocomotionRate`: en reposo respeta el ritmo del Idle. En movimiento escala entre 0.35 y 2.5 según la velocidad real, contra 5 m/s de frente o 4.4 m/s de lado o atrás (:381-386).
- La prioridad de estado de la base sale de :388-398: Stun > Dash/DashBackward > Slide > Jump/AirJump (ventana `_jumpPoseTime`) > Fall (no grounded) > Land (ventana, solo con < 0.5 m/s) > Crouch > Locomotion.
- Eventos: `OnJump`, `OnAirJump` y `OnLanded` abren las ventanas. `OnHealthChanged`, si la vida baja, activa Hit. `OnPlayerDied` activa Death en Full Body (:672-694).
- Dash atrás: histéresis local $z < -0.35$ para entrar y < -0.2 para quedarse (:447). Solo el Dash adelante o lateral recibe redirección procedural de caderas y muslos (:457-524).

Upper body

`UpdateUpperBody` (:404). La prioridad es Hit > Flame (sostenido) > acción (Fire/Slash) > Aim. El peso de la capa sube a `_upperBodyWeight` solo si hay arma manual activa, Hit, acción o carga. Con Stun se multiplica $\times 0.4$.

Los eventos salen de `WeaponPresentationController.FeedbackEmitted` (:696), y los de armas en modo Automatic se ignoran:

Evento	Efecto
ShotFired	Fire (Slash si es <code>RotatingBlade</code>), ventana de pose, reinicio del estado, recoil encolado
SustainedFireStarted/Stopped	Flame on/off
ChargeStarted	<code>_charging</code> = sube el peso de la capa, sin estado propio (queda Aim)
ChargeCancelled / AmmoEmpty	corta carga y sostenido

Aim procedural (hacia el reticle)

`ApplyAim` (:587). Solo actúa si el peso de Upper Body es > 0.001 . Sin arma manual no hay aim.

- Origen en `hand.R`. Yaw y pitch del objetivo en espacio del jugador, con clamp y suavizado (`_aimSmoothTime`). `ResolveLimitedYaw` (:640) evita saltar de lado al cruzar $\pm 180^\circ$.
- Pecho (`spine.003`): recibe el 50 % del yaw y el 40 % del pitch, con límites propios, rotados en ejes del mundo (:609-613).
- Brazos: `AimArm` hace 2 iteraciones de `FromToRotation` de antebrazo a objetivo sobre el upper arm, limitadas por `_armCorrectionLimit` (:621-637). Sin IK ni Humanoid.
- Brazo de apoyo (izquierdo): peso `_supportArmWeight` $\times (1 \rightarrow 0.12)$ según el braceo de carrera).
- Pesos: Slash $\times 0.25$, Hit $\times 0.4$ (:607-608).
- Giro de cuerpo: si el yaw crudo supera `_bodyTurnThreshold` y no hay slide, dash ni stun, `PlayerMovement.RequestAimFacing(dir, _bodyTurnHoldTime)` (:602-604).
- Cabeza y cuello no apuntan al reticle. Solo reciben el balanceo de `ApplyArmedRunFollowThrough` (:559-561).

Huesos del prefab (vía `Configure`, `PlaceholderPlayerAnimationBuilder.cs:88`): `chest.spine.003`, `upper_arm/forearm/hand.R/.L`. Además se resuelven por nombre `spine`, `thigh/shin.L/.R`, `spine.004`, `spine.006` y `shoulder.L/.R` (`PlayerAnimationDriver.cs:213-227`).

Valores serializados (player.prefab:1265-1290 = defaults de código)

Campo	Valor	Campo	Valor
<code>_referenceMoveSpeed</code>	5	<code>_aimWeight</code>	1
<code>_referenceSideAndBackwardSpeed</code>	4.4	<code>_supportArmWeight</code>	0.85
<code>_velocityBlendTime</code>	0.08	<code>_chestYawLimit</code> / <code>_chestPitchLimit</code>	40° / 22°
<code>_stateBlendTime</code>	0.09	<code>_armYawLimit</code>	95°
<code>_jumpPoseTime</code> / <code>_landingPoseTime</code>	0.2 / 0.22	<code>_aimUpLimit</code> / <code>_aimDownLimit</code>	70° / 60°
<code>_upperBodyWeight</code>	1	<code>_armCorrectionLimit</code>	110°
<code>_actionBlendTime</code>	0.045	<code>_bodyTurnThreshold</code>	60°
<code>_firePoseTime</code>	0.28	<code>_aimSmoothTime</code> / <code>_bodyTurnHoldTime</code>	0.055 / 0.1
<code>_slashPoseTime</code>	0.55	<code>_recoilDegrees</code> / <code>_maximumRecoil</code>	3° / 7°
<code>_hitPoseTime</code>	0.34	<code>_recoilRecovery</code>	18 °/s

Recoil: se acumula por disparo (`_recoilDegrees` × intensidad, hasta el máximo) y se aplica en el frame siguiente, rotando los upper arms sobre `transform.right`. Con `ReducedMotion` de `WeaponPresentationController` se escala × 0.2 (:710-711). El retroceso cosmético del cañón (`WeaponRecoilFeedback`) va aparte y es inmediato.

Wearables

`PlayerWearableSurfaceFollower`: cada attachment arma un marco de 3 anclas (A, B, C), cada una un promedio ponderado de puntos en huesos. Opcionalmente suma un asiento exacto (`Seat`) o una corrección de profundidad por sondas de contacto, con un tope de `MaximumContactCorrection` = 0.08. Todo se reconstruye en absoluto en cada evaluación, sin acumular (`PlayerWearableSurfaceFollower.cs:84-121`). En el prefab hay 5 attachments, uno por `WeaponType` (0-4), con huesos base `spine`, `shoulder.R`, `forearm.L` y `spine.003 ×2` (`PlaceholderPlayerAnimationBuilder.cs:170-171`).

Enemigos

`EnemySkinnedAnimBuilder` (menú `Tools/ScrapWaves/Fix Enemy Skinned Animations (Chaser+Drone)` más un menú por tipo):

Familia	FBX	Controller (estado)	Prefabs	Escala
Chaser	<code>LarguiruchoRigWalk_anim.fbx</code>	<code>ChaserWalk.controller</code> (Walk)	Chaser, Chaser (variant), Chaser_Elite	visual ×1.57; root ×2 en variant/elite
Drone	<code>Avispa_Vuelo.fbx</code>	<code>DroneFly.controller</code> (Fly)	Drone, Drone (variant), Drone_Elite	visual y+1; root ×4 en elite
Slime	<code>Basurita_MiniSalto.fbx</code>	<code>SlimeHop.controller</code> (Hop)	EnemyPro, Slime (variant), Slime_Elite	conserva la pose previa, yaw 0

Pasos (`EnemySkinnedAnimBuilder.cs:183-385`):

1. Import Generic con todos los takes en loop y root bloqueado.

2. Controller de una capa y un estado, sin parámetros.
3. En el prefab destruye el visual viejo, instancia el FBX, copia los materiales y pone `updateWhenOffscreen = true`.
4. Animator con `AlwaysAnimate`, sin root motion, avatar del FBX.
5. Deshabilita los renderers placeholder (`Cylinder`, `drone`).
6. Solo en Slime agrega `EnemyAnimPhaseOffset`.

`EnemyAnimPhaseOffset.OnEnable` ejecuta `Animator.Play("Hop", 0, Random.value)` (`EnemyAnimPhaseOffset.cs:12-19`). Un Play por spawn o salida del pool, sin Update. Está en `EnemyPro`, `Slime (variant)` y `Slime_Elite`.

Deuda / puntos abiertos

- Chaser y Drone no tienen `EnemyAnimPhaseOffset`: todas las instancias caminan y vuelan en fase. El componente además tiene hardcodeado el estado "Hop" (`EnemyAnimPhaseOffset.cs:10`), así que no se puede reusar en `ChaserWalk/DroneFly` sin generalizarlo.
- Todos los enemigos skineados quedan en `AlwaysAnimate` con `updateWhenOffscreen = true` (`EnemySkinnedAnimBuilder.cs:330, :355`). Con el swarm, el costo de skinning no se descarta fuera de cámara. Falta perfilarlo.
- El menú dice "(Chaser+Drone)" pero también reconstruye Slime (`EnemySkinnedAnimBuilder.cs:42-50`). `FixSlimeFacing (:390)` no tiene `MenuItem`: solo se llama por código.
- `ChargeStarted` sube el peso de Upper Body pero no hay estado de carga en el controller (`PlayerAnimationDriver.cs:416, :720-723`). La carga se ve como Aim.
- Las armas en modo Automatic no disparan pose de acción ni recoil óseo (`PlayerAnimationDriver.cs:698`). Sin arma manual, el peso de Upper Body queda en 0 y no hay aim de brazos (`:421, :589`).
- Los huesos auxiliares se buscan por nombre literal (`:213-227`). Si se renombra el rig, quedan en null en silencio: se apaga el dash direccional y el braceo, sin warning.
- `PlayerAutoAttack` sigue en el prefab deshabilitado (`player.prefab:619`) y llama `EvaluatePoseForWeapons` con la posición del enemigo (`PlayerAutoAttack.cs:53`). Si alguien lo habilita, compite con `WeaponManager` por la única evaluación del frame.
- El rig, el grafo y el controller siguen siendo placeholder ("`Player placeholder animation`", `:278`), con compresión de animación Off (`PlaceholderPlayerAnimationBuilder.cs:45`). Hay que revisar el costo de memoria antes del arte final.

3 Rear threat

Indicador de amenaza trasera: una media luna en el piso a la altura de la cadera del jugador que se estira en púas hacia los grupos de enemigos que vienen por detrás. No es UI de Canvas: es un mesh en mundo que se dibuja con `ZTest Always`, así que se ve a través del jugador y de las paredes. Separa lectura (sensor) de dibujo (presenter). El sensor no emite eventos de combate ni toca vida.

No hay otro widget "sensor" de enemigos: en `Assets/Scripts/Enemy` y `Assets/Scripts/Weapon/UI` el único `*Sensor*` es `RearThreatSensor`.

Scripts

Script	Rol
<code>Assets/Scripts/Weapon/UI/RearThreat/RearThreatSensor.cs</code>	Muestrea <code>EnemyRegistry</code> , filtra, agrupa y arma el snapshot. No crea UI
<code>Assets/Scripts/Weapon/UI/RearThreat/RearThreatSnapshot.cs</code>	<code>RearThreatSpike</code> y <code>RearThreatSnapshot</code> : copia inmutable, hasta 5 púas
<code>Assets/Scripts/Weapon/UI/RearThreat/RearThreatMesh.cs</code>	Geometría: tira radial de 128 segmentos con púas insertadas en el borde exterior
<code>Assets/Scripts/Weapon/UI/RearThreat/RearThreatPresenter.cs</code>	<code>[ExecuteAlways]</code> , orden 110. Maneja el sensor, suaviza con resortes, sigue al jugador, colorea
<code>Assets/Shaders/UI/RearThreatCrescent.shader</code>	Unlit URP, color plano <code>_BaseColor</code> , alpha blend, <code>ZWrite Off</code> , <code>ZTest Always</code> , <code>Cull Off</code> (:6-13)
<code>Assets/Scripts/Editor/RearThreatIndicatorAuthoring.cs</code>	Menús de autoría: prefab, materiales, meshes de preview e instancia bajo UI
<code>Assets/Scripts/Editor/RearThreatValidation.cs</code>	Compila player scripts y renderiza previews PC/Mobile a <code>Library/CodexValidation/RearThreat</code>
<code>Assets/Scripts/Editor/GameplayUiSceneMigration.cs</code>	La migración de UI llama <code>RearThreatIndicatorAuthoring.AuthorUi(ui, player)</code> (:132)

Assets: `Assets/Prefabs/UI/RearThreatIndicator.prefab` (raíz con sensor + presenter, hijos `Fill` y `Outline`), `Assets/Art/UI/RearThreat/RearThreatFill.mat` (cola 2991), `RearThreatOutline.mat` (cola 2992), `RearThreatFillPreview.asset` y `RearThreatOutlinePreview.asset` (meshes estáticos solo para el editor).

Qué cuenta como amenaza trasera

Fuente: `EnemyRegistry.CollectActive` (`Assets/Scripts/Enemy/EnemyRegistry.cs:46`), que ya descarta enemigos dormidos por altura (`EnemyVerticalEngagement.IsDisengaged`, :62). Sobre esa lista, el sensor (`RearThreatSensor.cs:266-303`) exige:

1. Que la raíz no sea el jugador ni un hijo suyo.
2. Vivo y activo: `GameObject` activo, `EnemyRegistryMember` habilitado si existe, y `EnemyHealth.CurrentHealth > 0`, o `WeaponDummyEnemy.CurrentHealth > 0` en el sandbox (:332-337).
3. Diferencia vertical $|\Delta y| \leq 3 \text{ m}$. Estricta, sin histéresis (:283).
4. Distancia planar (XZ, desde la raíz del enemigo, no desde sus bounds) $\leq 10 \text{ m}$, o $\leq 10,5 \text{ m}$ si ya era elegible (:281).
5. Dentro del arco trasero: ángulo contra `-forward` $\leq 90^\circ$, o $\leq 95^\circ$ si ya era elegible (:263-264, :290).

El `forward` sale del jugador (`PlayerFacing`, el valor serializado) o de la cámara (`CameraFacing`: usa `ThirdPersonCamera.GameplayForward` y, si no hay, `camera.forward`). Siempre se aplan a XZ (:227-255).

El jugador es válido si existe, está activo, tiene posición finita y `PlayerHealth.IsAlive` (:215-219). Si se destruye, el sensor intenta rebindear a `PlayerMovement.PlayerTransform` de la misma escena (:183-188).

Valores (prefab = escena)

`GameplayScene` no sobrescribe ningún valor numérico del sensor ni del presenter: solo `_player`, `_cameraReference` y `_gameplayCamera` (`GameplayScene.unity:20487-20498`). Los valores del prefab coinciden con los defaults del código.

Sensor (`RearThreatSensor.cs:24-35`, `RearThreatIndicator.prefab`):

Campo	Valor	Efecto
<code>_referenceFrame</code>	<code>PlayerFacing</code>	Base del arco
<code>_detectionRadius</code>	10 m	Radio de entrada
<code>_fullStrengthDistance</code>	2 m	Urgencia 1 a esta distancia o menos
<code>_rearArcDegrees</code>	180°	Arco total de entrada (±90°)
<code>_maxVerticalDifference</code>	3 m	Filtro de altura
<code>_sampleInterval</code>	0,1 s	Muestreo completo cada 100 ms de <code>Time.time</code>
<code>_distanceExitMargin</code>	0,5 m	Histéresis de salida por distancia
<code>_arcExitMargin</code>	5°	Histéresis de salida por borde de arco
<code>_clusterAngle</code>	15°	Radio angular de un grupo
<code>_maxSpikes</code>	5	Tope de púas (tope duro <code>RearThreatSnapshot.MaximumSpikes = 5</code>)
<code>_replacementDistanceAdvantage</code>	0,5 m	Ventaja que necesita un grupo nuevo para desplazar al más lejano

Presenter (`RearThreatPresenter.cs:13-40`):

Campo	Valor	Efecto
<code>_hipOffset</code>	(0; 0,25; 0)	Offset desde la raíz del jugador, rotado con el forward
<code>_innerRadius</code>	0,8 m	Hueco alrededor del jugador
<code>_idleOuterRadius</code>	0,86 m	Borde exterior en reposo (banda de 6 cm)
<code>_maximumRadiusMultiplier</code>	1,1	Borde exterior con urgencia 1: 0,946 m
<code>_minimumSpikeLength</code> / <code>_maximumSpikeLength</code>	0,1 / 0,65 m	Largo de púa según urgencia del grupo
<code>_spikeWidthDegrees</code>	10°	Ancho de la punta
<code>_shoulderWidthDegrees</code>	44°	Ancho del "hombro" redondeado
<code>_localFlex</code>	0,75	Parte del crecimiento que se concentra cerca de las púas
<code>_outlineWidth</code>	0,03 m	Trazo del contorno
<code>_growthTime</code> / <code>_relaxTime</code>	0,12 / 0,25 s	<code>SmoothDamp</code> al crecer / al relajar
<code>_directionSmoothingTime</code>	0,08 s	Seguimiento del rumbo de cada púa
<code>_idleColor</code>	(1; 0,35; 0,65; 0,08)	Rosa casi transparente en reposo
<code>_alertColor</code>	(1; 0,02; 0,06; 0,95)	Rojo casi opaco con urgencia 1

Campo	Valor	Efecto
<code>_outlineColor</code>	<code>(0; 0; 0; 0)</code>	Contorno apagado

Con un contorno de alpha 0, `SetVisible` deja apagado el renderer `Outline` (:337). En el prefab ese `MeshRenderer` ya viene con `m_Enabled`: 0.

Geometría

`RearThreatMesh.Update` (`RearThreatMesh.cs:26-116`):

- Ángulos en grados relativos a "atrás". Positivo = izquierda. `Direction(a) = (-sin a, 0, -cos a)` (:128-132).
- Medio arco dibujado = `rearArc/2 + arcExitMargin = 95°` (`RearThreatPresenter.cs:240`). El arco visible total es de 190°, 10° más ancho que el de entrada.
- 129 muestras uniformes, más 3 por cada púa (flanco, punta, flanco), ordenadas y sin duplicados.
- Radio exterior por muestra = `outer - crecimiento_localizado × hombro_no_cubierto + largo_púa`. Una púa corta sale como bulto redondeado y se afila al madurar (:55-88).
- Tapas en los extremos que no invaden el hueco interior. Si el arco llega a 360° no hay tapas (:98-112).
- Bounds fijos: $2 \times (\text{idleOuter} \times 1,1 + 0,65)$ de lado.

Color: `Color.Lerp(idle, alert, urgenciaSuavizada)` vía `MaterialPropertyBlock` sobre `_BaseColor`. El material compartido no cambia (`RearThreatPresenter.cs:254-264`). Los renderers no proyectan ni reciben sombras, no usan probes ni motion vectors (:274-283).

Flujo por frame

1. `RearThreatPresenter.LateUpdate` (:115) en play: si falta el sensor o los meshes, resetea y oculta.
2. `RearThreatSensor.RefreshForPresentation(Time.time)` (:174). Si pasó `_sampleInterval`, hace un muestreo completo (`Sample`). Si no, solo saca candidatos que murieron o se desactivaron y reagrupa (:204-208). El forward se recalcula cada frame.
3. `Sample` ordena candidatos por distancia (desempata por `InstanceID`) y llama `BuildGroups`.
4. `BuildGroups` (:358-464):
5. Agrupa con semilla = el más cercano sin grupo. Suma a todos los que quedan a $\leq 15^\circ$ de la semilla. No encadena.
6. Reasigna `TrackId` previos por solapamiento de miembros. Si cambia el enemigo más cercano, el track visual no cambia.
7. Primero retiene los grupos con track. Después completa hasta 5. Un grupo nuevo desplaza al seleccionado más lejano solo si está 0,5 m más cerca.
8. Urgencia de cada púa = `InverseLerp(10, 2, distancia del grupo)`. `OverallUrgency` = urgencia del candidato más cercano, esté o no en una púa seleccionada.
9. El presenter empareja púas por `TrackId`. Un track nuevo arranca en largo 0 y en su propio rumbo, sin barrer desde el anterior (:226-238). Una púa que quedó fuera del arco por un giro va a largo 0 (:216).
10. `Follow`: pone el objeto en `player.position + rot × hipOffset` y compensa la escala del padre `UI` (:242-250).
11. `Draw` regenera los meshes `fill` y `outline`.

`EnemyRegistry.Unregistered` invalida en el acto al enemigo despawneado (:495-512). Con el pool, la reutilización arranca con track nuevo.

Pausa: con `Time.deltaTime == 0` (pausa de UI o hit-stop) los resortes se congelan. Solo se reexpresan los rumbos si cambia la base (:155-162). Como `Time.time` no avanza, tampoco se remuestra.

Autoría y editor

Menús en ScrapWaves/UI/:

Menú	Hace
Create Missing Rear Threat Indicator Assets	Crea el prefab, los materiales (colas 2991/2992) y los meshes de preview si falta el prefab (<code>RearThreatIndicatorAuthoring.cs:35-76</code>)
Add Rear Threat Indicator To Current Scene	Instancia bajo la raíz UI y bindea jugador y cámara <code>MainCamera</code> (:78, :137-201)
Refresh Rear Threat Prefab Preview Meshes	Regenera los assets de preview desde el prefab (:81-100)
Add Rear Threat Indicator To All Player Scenes	Backup a <code>Library/RearThreatIndicatorAuthoring/<fecha></code> y autoría en 6 escenas fijas (:25-33, :203-233)

`AuthorUi` no pisa lo que ya existe. Falla si hay dos indicadores, si el indicador existente apunta a otro jugador o si la jerarquía está incompleta.

En modo edición, el presenter dibuja un preview con `_previewUrgency`, `_previewSpikeAngles` y `_previewSpikeUrgencies` (:372-394). `RearThreatPreviewSaveProcessor` (:298-319) restaura los meshes autorados antes de cada guardado y vuelve a generar el preview en el `delayCall`, para que el mesh dinámico (`HideFlags.DontSave`) no termine serializado.

Validación (`ScrapWaves/Validation/Rear Threat/`): *Compile and Render y Render PC and Mobile Previews* (`RearThreatValidation.cs:63-165`). Renderiza 6 casos (idle, single, cluster, a través de pared, cámara baja, jugador real) en los perfiles de calidad PC y Mobile. `BeginPlayModeValidation` (:30-43) no tiene menú: es para batch y corre `RearThreatPlayModeValidationTests`.

Escena

`GameplayScene.unity:20443-20515`: instancia de `RearThreatIndicator.prefab` bajo la raíz UI (`GameplayScene.unity:30161`, posición mundial (238,55; 0; 34,09)). `_player` apunta al Transform de `player.prefab`; `_cameraReference` y `_gameplayCamera`, a la cámara principal. El indicador también está en `SampleScene`, `WeaponTestingSandbox`, `WeaponTestingSandbox_GameFeel`, `test_balance` y `enemiesTesting`.

Tests

`Assets/Tests/Editor/`: `RearThreatSensorTests` (filtros, histéresis, clustering, tracks, 0 allocs con 100 enemigos), `RearThreatPresentationTests` (continuidad del mesh, resortes, pausa, paleta sin tocar materiales), `RearThreatAuthoringTests` (preserva ediciones manuales, un indicador por escena de jugador) y `RearThreatPlayModeValidationTests` (Play Mode, transiciones y allocs en régimen).

Deuda / puntos abiertos

- `GameplayScene` sobrescribe `m_Mesh` de `Fill` y `Outline` a `None` (`GameplayScene.unity:20484`, :20504). Lo mismo pasa en `WeaponTestingSandbox` y `WeaponTestingSandbox_GameFeel`. En runtime no afecta, porque `EnsureMeshes` crea meshes dinámicos. Pero contradice `RearThreatAuthoringTests.PlayerScene_HasOneAuthoredRearThreatIndicatorUnderUi` (`RearThreatAuthoringTests.cs:150`, `AssertMesh` exige `sharedMesh` persistente no nulo): en esas tres escenas ese test debería fallar. Hay que revertir el override.
- El presenter regenera los dos meshes en cada `LateUpdate` aunque no haya amenazas (`RearThreatPresenter.cs:148`). También genera el `outline` con el contorno apagado (alpha 0 y renderer deshabilitado). Es trabajo por frame que se puede evitar con dirty-check.

- `MaximumSpikes = 5` está duplicado en `RearThreatSnapshot.cs:26` y `RearThreatMesh.cs:6`, sin vínculo entre las dos constantes. El sensor usa una y el presenter la otra.
- `OverallUrgency` (el color de toda la media luna) toma al enemigo más cercano aunque su grupo no esté entre las púas mostradas (`RearThreatSensor.cs:462`). Puede haber rojo sin púa visible en esa dirección.
- Hay dos listas de escenas distintas. El menú de rear threat usa una lista fija de 6 (`RearThreatIndicatorAuthoring.cs:25-33`). `GameplayUiSceneMigration` busca escenas por el GUID del jugador (`GameplayUiSceneMigration.cs:32-34`). Una escena de jugador nueva solo entra por la segunda vía.
- El arco dibujado ($\pm 95^\circ$) es más ancho que el de entrada ($\pm 90^\circ$) porque suma la histéresis (`RearThreatPresenter.cs:240`). Si se quiere que el borde visual marque el límite real de detección, hay que separar los dos valores.
- La salida de validación va a `Library/CodexValidation/...` (`RearThreatValidation.cs:19`), un nombre heredado de otra herramienta.

PARTE 2

Combate y presentación

4 Daño

Hay dos caminos separados. El daño **del jugador hacia enemigos** pasa por un contrato tipado (`DamageRequest` → `IDamageable` → `DamageApplicationResult`) y siempre entra por `WeaponDamageApplier`. El daño **de enemigos hacia el jugador** no usa ese contrato: cada behavior llama `PlayerHealth.TakeDamage(int)` directo. Los dos lados comparten solo la idea de i-frames/invencibilidad y los hooks de audio y challenges.

Scripts

Script	Rol
<code>Assets/Scripts/Damage/DamageRequest.cs</code>	Struct inmutable del pedido: daño pedido, daño modificado, canal, status, lifesteal, arma origen
<code>Assets/Scripts/Damage/DamageChannel.cs</code>	Direct / Status
<code>Assets/Scripts/Damage/DamageApplicationResult.cs</code>	Resultado: aplicado, bloqueado, kill, vida antes/después
<code>Assets/Scripts/IDamageable.cs</code>	<code>IDamageable</code> (legacy, <code>bool ApplyDamage(int)</code>), <code>IAuthoritativeDamageable</code> y la extensión que despacha
<code>Assets/Scripts/Weapon/Projectiles/WeaponDamageApplier.cs</code>	Punto único de entrada del daño de armas
<code>Assets/Scripts/Weapon/Projectiles/WeaponDamageAmplifierStatus.cs</code>	Estado Vulnerable: multiplica el daño recibido
<code>Assets/Scripts/Weapon/Managers/WeaponDamageResolver.cs</code>	Calcula el número (nivel, path, stats, crit, elite, rango)
<code>Assets/Scripts/Weapon/Projectiles/WeaponRadialDamage.cs</code>	Daño en área con falloff
<code>Assets/Scripts/Enemy/EnemyHealth.cs</code>	Vida del enemigo, invencibilidad, muerte y despawn
<code>Assets/Scripts/Enemy/EnemyDamageHitZone.cs</code>	Zona de hit con multiplicador propio (cabeza)
<code>Assets/Scripts/Enemy/Behaviors/DestroyerMouthWeakPoint.cs</code>	Punto débil con vida propia del Destroyer
<code>Assets/Scripts/Player/PlayerHealth.cs</code>	Vida del jugador, i-frames, escudo, burn, regeneración
<code>Assets/Scripts/Enemy/EnemyContactDamage.cs</code> + <code>Player/PlayerContactHurtbox.cs</code> / <code>PlayerContactDamageReceiver.cs</code>	Daño por contacto
<code>Assets/Scripts/Enemy/EnemyOutgoingDamageScale.cs</code>	Escala del daño saliente del enemigo

Contrato

`DamageRequest` (`DamageRequest.cs:3-26`) clampa a ≥ 0 ambos valores. Defaults del constructor: `statusKind = Burn`, `canTriggerLifesteal = true`, `sourceWeaponId = null`.

Campo	Significado
<code>RequestedDamage</code>	Daño que calculó el arma
<code>ModifiedDamage</code>	Tras Vulnerable. Es lo que el receptor usa
<code>Channel</code>	Direct (impacto) o Status (DoT)
<code>StatusKind</code>	Tipo de estado, para feedback
<code>CanTriggerLifesteal</code>	Si el daño aplicado cura al jugador
<code>SourceWeaponId</code>	Crédito de kill para challenges

`DamageApplicationResult` tiene cuatro fábricas (`DamageApplicationResult.cs:37-99`):

Fábrica	Cuándo	Notas
<code>Rejected</code>	Daño ≤ 0 o target ya muerto	<code>Applied = false</code> , <code>Blocked = false</code>
<code>BlockedResult</code>	Invencibilidad	<code>Blocked = true</code>
<code>FromHealthDelta</code>	Caso normal	<code>AppliedDamage = antes - después</code> . Killed si llega a 0
<code>FromLegacy</code>	Target que solo implementa <code>IDamageable</code>	<code>IsAuthoritative = false</code> , vida 0/0, nunca Killed

`ModifiedDamage` del resultado es el del pedido, no el escalado por el receptor. El daño real está en `AppliedDamage`.

La extensión `IDamageable.ApplyDamage(in DamageRequest)` (`IDamageable.cs:22-32`) usa la versión autoritativa si existe. Si no, cae al `bool ApplyDamage(int)` con `ModifiedDamage`.

Implementan `IAuthoritativeDamageable`: `EnemyHealth`, `EnemyDamageHitZone`, `DestroyerMouthWeakPoint`, `WeaponDummyEnemy` y `WeaponDummyWeakPoint` (sandbox). No hay implementaciones solo legacy en producción.

Flujo: arma → enemigo

1. El arma resuelve el `IDamageable` con `GetComponentInParent<IDamageable>()` sobre el collider golpeado. Toma primero el objeto golpeado, así que una hit zone o un weak point le gana al `EnemyHealth` del padre.
2. Calcula el número con `WeaponDamageContext.CalculateDamageValue` (`WeaponDamageResolver.cs:306-346`): `TargetNeutralDamage × elite × rango × DamageScale × escalaAdicional`, redondeado con mínimo 1. Rango: más de 15 m usa `LongRangeDamageMultiplier` y menos de 10 m `CloseRangeDamageMultiplier` (:300-305).
3. Llama `WeaponDamageApplier.ApplyDamage` (`WeaponDamageApplier.cs:8-44`): - `ModifiedDamage = max(1, round(daño × Vulnerable))` (:21-23, `WeaponDamageApplierStatus.cs:32-42`). - Arma el `DamageRequest` y despacha. - Notifica el arma a `EnemyHealth.NotifyDamagedByWeapon` (:33-34). - Si `AppliedDamage > 0`: `ChallengeProgressTracker.NotifyDamageInstance` y, si corresponde, `PlayerCombatHooks.TryLifesteal` (:36-41).
4. `EnemyHealth.ApplyDamageInternal` (`EnemyHealth.cs:127-163`): - Rechaza si el daño es ≤ 0 o la vida ya es 0. - Bloquea si es invencible: `Direct` siempre, `Status` solo si se activó con `blockDot` (:133-135). - Escala: `round(ModifiedDamage × multiplicador)`. Usa `_damageTakenMultiplier` (1 por defecto) o el de la hit zone (:139). - Si mata: SFX de muerte, challenge, `OnDied`, `RunCombatStats` y despawn al pool o `Destroy` (:165-185). Si no mata y es `Direct`: SFX de hit (:157-160).

Llamadores de `WeaponDamageApplier.ApplyDamage`: `Projectile` (impacto, explosión, cono de fragmentos), `MortarShellImpact`, `WeaponRadialDamage`, `AutomaticCannonWeapon`, `FlamethrowerWeapon`, `FlamethrowerBurnStatus`, `FlamethrowerFuelPuddle` (vía radial, canal `Status`) y `RotatingBladeWeapon`.

Área

`WeaponRadialDamage.Apply` (`WeaponRadialDamage.cs:26-60`) deduplica por `IDamageable`. Aplica el falloff por distancia al centro del collider:

```
escala = lerp(1, 1 - falloff, distancia / radio)
```

Por defecto corta en 128 targets. Los buffers de colliders van de 256 a 4096.

Zonas especiales

Target	Valores	Comportamiento
EnemyDamageHitZone	×1,2 por defecto (EnemyDamageHitZone.cs:9)	Reenvía a EnemyHealth.ApplyHitZoneDamage. Reemplaza _damageTakenMultiplier, no lo multiplica
DestroyerMouthWeakPoint	80 HP (DestroyerMouthWeakPoint.cs:14)	Vida propia. No mira la invencibilidad del cuerpo. A 0 dispara OnWeakPointDestroyed y se desactiva
Weak point por nombre	—	Projectile y AutomaticCannonWeapon detectan un collider cuyo nombre contiene "WeakPoint" (Projectile.cs:1152-1158, AutomaticCannonWeapon.cs:1185-1188). Afecta el feedback y el bonus Head Hunter

Invencibilidad del enemigo

SetInvincible(bool, blockDot) (EnemyHealth.cs:28-32). Por defecto el DoT sigue pasando. blockDot: true da inmunidad total, por ejemplo al Destroyer durante la succión. OnEnable y PrepareForPoolSpawn resetean vida, invencibilidad y crédito de arma (:50-75).

Flujo: enemigo → jugador

1. El behavior calcula el daño. Casi todos pasan por EnemyOutgoingDamageScale.ScaleFrom (EnemyOutgoingDamageScale.cs:30-47). Usa el multiplicador cacheado al spawn y, si no hay componente, DifficultyManager.GetEnemyDamageMultiplier().
2. Llama PlayerHealth.TakeDamage(int) (PlayerHealth.cs:282-325). Lo hacen EnemyContactDamage, EnemyProjectile, EnemySeekingMissile, EnemyC4, FireArea, CorrosiveSlimeArea, ChargerEnemyBehavior, HellfireSlimeBehavior, DestroyerBehavior (tragar) y GigaWormBehavior.
3. TakeDamage ignora el golpe en estos casos, en orden: - Daño ≤ 0 o jugador muerto. - Invulnerabilidad de powerup (:287). - i-frames activos (:290).
4. Si hay escudo, consume una carga y abre los i-frames. No resta vida (:293-302).
5. Si no hay escudo: - Resta max(1, round(daño × (1 - resistencia))). La resistencia tiene tope 0,95 (PlayerStatMath.cs:5-12). - Abre los i-frames, reinicia el delay de regeneración y dispara SFX de hurt, OnHitDamageTaken y OnHealthChanged. - A 0 HP dispara OnPlayerDied una sola vez.
6. Durante los i-frames, SyncEnemyCollisionIgnores ignora la colisión física con los enemigos activos (:359-368).

El contacto entra por dos vías: el trigger del hurtbox (PlayerContactHurtbox) o colisiones del Rigidbody (PlayerContactDamageReceiver, con OnCollisionStay). Las dos terminan en EnemyContactDamage.TryApplyContactDamage: 5 de daño base y empuje 3 (EnemyContactDamage.cs:5-8).

Valores del jugador

Campo	Script	Prefab	
_maxHealth	100	100 (player.prefab:654)	—
_hitInvulnerabilitySeconds	1,5	1,5 (:656)	0,5 (override, GameplayScene.unity:34442)
_regenerationDelaySeconds	5	5 (:658)	—

Burn del jugador

ApplyBurn(segundos, dps) (PlayerHealth.cs:220-229) no apila: se queda con la mayor duración y el mayor dps. Ticea cada 0,5 s con max(1, round(dps × 0,5)) y aplica resistencia (:231-251). No respeta i-frames, escudo ni invulnerabilidad de powerup. Tampoco dispara SFX ni OnHitDamageTaken.

Deuda / puntos abiertos

- El jugador no es `IDamageable`. El daño entrante no tiene canal, resultado ni fuente, así que no hay crédito de muerte por tipo de enemigo ni API común para feedback.
- **El DoT de armas cura con lifesteal.** `EnemyHealth.ApplyDotDamage` fija `canTriggerLifesteal: false` (`EnemyHealth.cs:96-105`), pero no tiene llamadores. El burn real pasa por `WeaponDamageApplier` con canal `Status` y deja el default `true` (`FlamethrowerBurnStatus.cs:153-158`).
- `WeaponDamageApplier.PendingSourceWeaponId` y `SetPendingWeapon/ClearPendingWeapon` (`WeaponDamageApplier.cs:6,51-64`) no tienen llamadores. Hoy el crédito depende de que cada arma pase `sourceWeaponId`.
- El crédito de arma se anota aunque el golpe haya sido rechazado o bloqueado (`WeaponDamageApplier.cs:33-34`). Un hit contra un enemigo invencible cambia `LastDamagingWeaponId`.
- Vulnerable queda en el objeto golpeado (`GetComponent`, no `InParent`, en `WeaponDamageAmplifierStatus.cs:25,37`). Si se aplica sobre una hit zone o el weak point, no amplifica el daño al cuerpo, y al revés.
- El weak point se detecta por substring de nombre (`"WeakPoint"`) en dos armas. Renombrar un collider rompe el bonus sin error.
- El burn del jugador ignora el escudo y la invulnerabilidad de powerup (`PlayerHealth.cs:231-251`, contra `:287,293`).
- Los i-frames de la escena (0,5 s) difieren del prefab y del script (1,5 s). Falta documentar cuál es el valor de diseño.
- `FromLegacy` devuelve vida 0/0 y nunca `killed`. Sin implementaciones legacy vivas es inocuo, pero cualquier `IDamageable` nuevo que no sea autoritativo va a romper el feedback de kill sin avisar.

5 Feedback de combate

El game feel de combate lo resuelve una sola capa de presentación, que el gameplay alimenta con eventos inmutables (`WeaponFeedbackContext`). Ninguna pieza de esta capa lee ni escribe vida. Los canales son VFX, audio, impulso de cámara, hit-stop, recoil, números de daño y reacción del enemigo (flash, empuje, estados, muerte). Cada canal tiene su propio rate limit y respeta las opciones de accesibilidad.

Scripts

Script	Rol
<code>Assets/Scripts/Weapon/Presentation/WeaponPresentationController.cs</code>	MonoBehaviour dueño, en el player. Implementa <code>IWeaponFeedbackSink</code> y crea un director por perfil
<code>Assets/Scripts/GameFeel/CombatFeedbackDirector.cs</code>	Ruteo de un perfil: cues, loops, cámara, hit-stop, recoil, texto, reacción
<code>Assets/Scripts/GameFeel/WeaponFxDirector.cs</code> / <code>WeaponAudioDirector.cs</code>	Pools de VFX por cue y voces de audio por director
<code>Assets/Scripts/GameFeel/CameraFeedbackController.cs</code>	Escala y limita el impulso hacia <code>ThirdPersonCamera</code>
<code>Assets/Scripts/GameFeel/HitStopController.cs</code>	Congela <code>Time.timeScale</code> con prioridad y replay mínimo
<code>Assets/Scripts/GameFeel/WeaponRecoilFeedback.cs</code>	Retroceso cosmético del cañón y tinte de calor
<code>Assets/Scripts/GameFeel/ProjectileVisualController.cs</code>	Mesh, material, escala y emisión del proyectil según arquetipo y calor
<code>Assets/Scripts/GameFeel/ImpactSurfaceResolver.cs</code>	Superficie de impacto: metadata, layer <code>Terrain</code> o nombre de material
<code>Assets/Scripts/GameFeel/CombatText/*</code>	Números de daño: evento, agregación, política, pool, vista, formato
<code>Assets/Scripts/GameFeel/EnemyHitFeedback.cs</code>	Flash (shell) y squash/empuje del visual del enemigo
<code>Assets/Scripts/GameFeel/EnemyDeathFeedback.cs</code> / <code>EnemyDeathReactionVfx.cs</code>	Snapshot y efecto de muerte desacoplado del despawn
<code>Assets/Scripts/GameFeel/EnemyStatusFeedback.cs</code>	Anillos de estado (burn, jellified, slow, freeze, vulnerable)
<code>Assets/Scripts/GameFeel/EnemyReactionProfile.cs</code>	Perfil de reacción más <code>EnemyReactionRuntime</code> (opciones globales estáticas)
<code>Assets/Scripts/GameFeel/GameFeelQualitySettings.cs</code>	Multiplicadores de partículas, luces y decals por calidad
<code>Assets/Scripts/Settings/Accessibility/*</code>	<code>PresentationAccessibilitySettings</code> / <code>State</code> / <code>Runtime</code>

Assets en uso

Asset	Referenciado desde
<code>Assets/ScriptableObjects/GameFeel/CombatTextProfile.asset</code>	<code>player.prefab:1040</code> (<code>_combatTextProfile</code>)
<code>Assets/ScriptableObjects/WeaponPresentation/{AutomaticCannon, Flamethrower, Mortar, RocketLauncher, RotatingBlade}Presentation.asset</code>	<code>WeaponData.PresentationProfile</code> de cada arma (<code>_profile</code> del controller = null, <code>player.prefab:1036</code>)
<code>Assets/GameFeel/Profiles/GameFeelQuality_PC.asset</code>	los 5 perfiles de presentación

Asset	Referenciado desde
Assets/GameFeel/Resources/EnemyReactionProfile.asset	Resources.Load EN EnemyReactionProfile.Resolve (EnemyReactionProfile.cs:81-93)
Assets/GameFeel/Prefabs/CombatText/CombatTextView.prefab	CombatTextProfile.ViewPrefab

Entradas y ruteo

WeaponPresentationController ([DefaultExecutionOrder(-40)]) arma un CombatFeedbackDirector por cada WeaponPresentationProfile distinto (WeaponPresentationController.cs:512-541). Todos comparten las mismas GameFeelRuntimeOptions, el CameraFeedbackController, el HitStopController y un único CombatTextDirector.

Entrada	Qué hace
Emit(WeaponPresentationCon text) (legacy, con cue explícito)	EmitLegacy → PlayCue con hit-stop y reacción de enemigo (CombatFeedbackDirector.cs:80-90). Lo usan los impactos de Projectile, MortarShellImpact, el impacto final de Blade y los cues propios de Cannon y Rocket
EmitSemantic(evento, contexto)	resuelve el cue por bindings (evento, modo, path, crit, weak, kill, superficie) (:136-196)
Begin/Update/EndSemanticLo op, Begin/Update/EndLegacyLoop	loops (flamethrower, cannon continuo, targeting de rocket)
OnDamageConfirmed	texto de daño + reacción del enemigo + hit-stop, si el perfil tiene binding
OnStatusSegmentClosed	cierra el tally de burn en el texto
ConfigureProjectile	aplica el arquetipo visual al proyectil (:242-264)

Flujo de EmitSemantic:

1. En un DamageConfirmed con kill, la intensidad se multiplica por EnemyDeathFeedback._effectIntensity (:142-144).
2. DamageConfirmed → CombatTextDirector.TryEmit. Si no es de la familia burn, también EnemyHitFeedback.TryPlay (:146-156).
3. ShotFired → cierra el loop de carga y pide recoil (:161-169).
4. Resuelve el cue. Si es loop o está rate-limited, sale.
5. Presentación (VFX, audio, cámara) solo si el evento no es DamageConfirmed, o si es un kill (:177-180).
6. Hit-stop solo para DamageConfirmed que no sea burn, con duración × intensidad (:182-192).
7. Si algo se reprodujo, _nextCueTimes[cue] = now + MinReplayInterval.

Update del controller: no corre con GameplayPause.IsUiPaused. Tickea FX y audio, y el hit-stop solo una vez. El texto no avanza si el timeScale es 0 por una pausa que no sea hit-stop (WeaponPresentationController.cs:446-461).

Hit-stop

HitStopController pone Time.timeScale = 0 y lo restaura con tiempo no escalado (HitStopController.cs:50-105).

Parámetro (player.prefab:1065-1068 = código)	Valor
_minimumReplayInterval	0.035 s

Parámetro (player.prefab:1065-1068 = código)	Valor
<code>_reducedFeedbackScale</code> (ReducedShake o ReducedFlash)	× 0.5
<code>_reducedMotionImportantDurationCap</code>	0.015 s. Con ReducedMotion solo pasan los importantes (kill, crit, weak point, ability)

- Un pedido de menor prioridad no pisa uno activo más largo.
- La duración efectiva es el máximo entre la activa y la nueva.

Duraciones y prioridades de hit-stop en los perfiles (cues con valor > 0):

Perfil	Cue (duración s / prioridad)
Cannon	CriticalImpact 0.014/1, WeakPointImpact 0.024/2, KillImpact 0.04/3
Rocket	Impact 0.02/2, KineticImpact 0.035/3, FragmentImpact 0.012/1, ClusterDetonation 0.025/3, KillImpact 0.045/4
Mortar	Impact 0.018/1, GrapeshotAirburst 0.006/1, MultiChargedImpact 0.014/1, MultiChargedRepeat 0.018/2
Blade	ManualSlash 0.02/0, ActiveThrust 0.06/1, MultiSlash 0.025/0, MultiThrust 0.045/1, AtomicSlash 0.018/0, AtomicDash 0.08/2, MultiFinalImpact 0.035/2, AtomicSliceImpact 0.012/1
Flamethrower	ninguno

Cámara (shake / kick)

`CameraFeedbackController.Request` (`CameraFeedbackController.cs:53-89`). La escala es `_masterScale` × intensidad × (0.25 si ReducedShake) × `heat.CameraVibration( calor )` × atenuación por distancia (llega a 0 a 28 m). La cámara (`ThirdPersonCamera.cs:189-221`) aplica reduced motion y los topes.

Valor	Fuente	Valor
<code>_masterScale</code> / <code>_reducedShakeScale</code>	player.prefab:1061-1062	1 / 0.25
<code>_minimumImpulseInterval</code> / <code>_maximumImpactDistance</code>	player.prefab:1063-1064	0.025 s / 28 m
Tope de posición / rotación / FOV	GameplayScene.unity:5430-5433	0.35 / 5° / 5
Decay	escena	12
ReducedMotion posición / rotación / FOV	escena	× 0.2 / × 0.35 / × 0

Rango de impulsos en los perfiles: de FOV 0.08 y rot -0.035 (cannon continuo) a FOV 2 y rot -1 (Rocket KillImpact). El impulso siempre es retroceso en z y pitch negativo.

Números de daño (CombatText)

1. `CombatTextEvent.TryFromFeedback` valida el contexto. Clave de agregación: target, arma, tipo (explosión y fragmento cuentan como una sola familia), secuencia, instancia de estado, segmento (`CombatTextAggregation.cs:9-90`).
2. Merge: con `ActionSequenceId > 0` se agrega hasta que `DamageFeedbackSequenceRuntime` marca la secuencia completa. Sin id, por ventana de fallback. Siempre con tope `*MaximumSegmentLifetime` (`CombatTextDirector.cs:321-346, :500-517`).
3. Prioridad y estilo (`CombatTextPresentationPolicy.cs:70-101`): Decorative 0, BurnTally 10, AutomaticDirect 20, ManualDirect 30, EliteBoss 40, MajorAbility 50, Critical 60, WeakPoint 70, CritWeak 80, Kill 90, CritWeakKill 100.
4. Visibilidad (`CombatTextDirector.cs:358-470`): modo (Off / ImportantOnly ≥ 40 / Full), en pantalla y delante de cámara, distancia, presupuesto de starts por frame (reserva uno para prioridad ≥ EliteBoss), densidad, y pool. Si

no alcanza, reclama una vista de menor prioridad (nunca $\geq$ WeakPoint).

5. 4 carriles verticales (0, +20, -20, +40 unidades de motion $\times$ 0.015 m) y sesgo de 0.05 m hacia la cámara.

6. Formato sin allocations (`CombatTextFormatter.cs`): entero completo hasta 99 999. Por encima, K/M/B/T con un decimal solo si la parte entera es < 10 .

7. `CombatTextWorldRenderDriver.LateUpdate` (orden 100) refresca los billboards de todo el pool después de la cámara.

`DamageFeedbackSequenceRuntime`: registro estático de capacidad fija (64, timeout huérfano 1.25 s). Lo abren Cannon, Mortar, Rocket y Blade (`BeginSequence`). En overflow devuelve id 0, que cae al fallback por ventana.

`StatusDamageSource` lleva arma, sink, modo, path, daño de referencia (≥ 1) e id de instancia de burn para que cada tick de burn se agregue en su tally.

Valores de `CombatTextProfile.asset`:

Grupo	Valores
Espacial	Sorting 800, WorldTextScale 0.10, 0.015 m por unidad de motion, sesgo a cámara 0.05
Estilos (tamaño / escala base)	Normal 34/1, Burn 30/0.9 (naranja), Jellified 30/0.9 (verde), Crit 40/1.06, Weak 38/1.07, CritWeak 44/1.1, Kill 46/1.1, Ability 36/1.04
Motion (vida s)	Normal 0.78, BurnTally 0.48, Crit 0.86, Weak 0.84, Kill 0.92, Reduced 0.66 (sin shake local)
Magnitud	curva ratio→escala 0.25:0.86 ... 8:1.38; clamp 0.85–1.42; resuelta 0.82–1.48; crit/weak $\times$ 1.08, tope CritWeak 1.16, kill $\times$ 1.05, elite/boss $\times$ 1.03, burn $\times$ 0.9
Fallback (s)	cannon auto 0.16 / manual 0.24 / scatter 0.18, HeadHunter 0.08, contacto 0.14 (blade 0.22), rocket 0.14, fragmento 0.30, llama 0.30, burn 0.65, mortar 0.18, multi-hit 0.18
Vida máxima de segmento (s)	directo 1.10, rocket 1.10, burn 3.25; gracia 0.12
Densidad Low/Med/High	prewarm 18/26/36, activas 16/24/32, starts por frame 3/5/7, burn visibles 6/10/16; pool máx. 40, agregados 128, secuencias 64
Distancia	tamaño completo 26 m, rutina 38 m, importantes 50 m, <code>DistantScaleMultiplier</code> 1
Ancla burn	<code>WorldAnchorHeight</code> 4.34 (código 1.25), clearance 0.25
Accesibilidad	lateral en <code>ReducedMotion</code> $\times$ 0.35, <code>ReducedShakeMultiplier</code> 0

Reacción del enemigo

`EnemyHitFeedback.TryPlay` busca el componente en el padre o en los hijos. Si no lo encuentra, lo agrega en caliente sobre `EnemyHealth` (`EnemyHitFeedback.cs:115-133`). Está serializado en los 14 prefabs de enemigo, con `_renderers` vacío.

- Tier (`EnemyReactionProfile.cs:95-107`): Kill > WeakPoint > Critical > Heavy (ability, ≥ 18 % de la vida máxima, o intensidad ≥ 1.15) > Light.
- Intensidad = evento $\times$ clase (elite 0.75, boss 0.42) $\times$ firma del arma. La firma es flamethrower 0.5, rocket 1.18, mortar 1.25, blade 0.82 y cannon 1 (`EnemyHitFeedback.cs:306`). Acumula con decay $\times$ 0.62 y tope 1.35.
- Flash: un "shell" duplicado de cada renderer (escala 1.012), tintado por `MaterialPropertyBlock`. Visibilidad $\times$ 0.35 con `ReducedFlash` y 0 si `ScreenFlash` está off (: 225-243).

<code>EnemyReactionProfile.asset</code>	Light	Heavy
Duración ($\times$ 1.25 si crit/weak)	0.085 s	0.14 s
Desplazamiento	0.055	0.12
Squash	0.055	0.12

Estados: hasta 3 visuales por enemigo y 48 globales, fade de 0.24 s. Muerte: pool de 48 y duración de 1.25 s. Colores: light (1, 0.34, 0.08, 0.72), crit (1, 0.84, 0.28, 0.92), weak blanco 0.95. El asset no serializa los tres campos de reduced motion, así que usan el default de código (desplazamiento 0.2, squash 0.25, duración 0.75).

Muerte: `EnemyDeathFeedback` se suscribe a `EnemyHealth.OnDied`, calcula bounds y color, y programa `EnemyDeathReactionVfx.Schedule`. Ese paso captura un snapshot del mesh (`BakeMesh` en los skineados). El efecto se spawna el frame siguiente: anillo, núcleo y 14 fragmentos con `LineRenderer`, en tiempo no escalado.

Accesibilidad y settings

Hay dos fuentes que convergen en `GameFeelRuntimeOptions` (player.prefab:1043-1059, todo en on, CombatText Full, Quality High):

Fuente	Campos	Llega por
<code>UserSettingsService</code> (menú principal, <code>SettingsScreenUI</code>)	<code>ReducedMotion</code> , <code>ScreenShake</code> , <code>ScreenFlash</code>	<code>UserSettingsApplier</code> → <code>ApplyUserFeedbackPreferences</code> (<code>WeaponPresentationController.cs:418-430</code>), <code>ThirdPersonCamera.ScreenShakeEnabled</code> , <code>EnemyReactionRuntime.ApplyUserPreferences</code>
<code>PresentationAccessibilityRuntime</code> (persistido por <code>SaveManager</code> , pausa <code>PauseMenuUI</code>)	<code>ReducedMotion</code> , <code>ReducedShake</code> , <code>ReducedFlash</code> , <code>CombatText</code> , <code>CombatTextScale</code> 0.75–1.25	evento <code>Changed</code> → <code>ApplyAccessibilityState</code> (:645-653) → <code>EnemyReactionRuntime.Apply</code>

Efecto de cada opción:

- `ReducedMotion`: bloquea el shake. Hit-stop solo para importantes, con tope de 15 ms. Recoil × 0.2, hueso y cañón. Cámara según la tabla. Texto con motion Reduced. Enemigo sin empuje ni squash.
- `ReducedShake`: cámara × 0.25, hit-stop × 0.5, sin shake local del texto.
- `ReducedFlash` / `ScreenFlash` off: flash del enemigo × 0.35 / 0, y reducción de flash en VFX.
- `CombatText`: Off libera todas las vistas.
- `GameFeelQualitySettings` (PC): partículas 0.35 / 0.7 / 1, decals 0.35 / 0.7 / 1, luces off en todos los niveles. La calidad cambia el pool del texto y los pools de VFX.

Deuda / puntos abiertos

- El hit-stop de los cues de `ShotFired` de Blade (0.018–0.08 s) no se aplica nunca. El camino semántico pasa `playHitStop: false` (`CombatFeedbackDirector.cs:180`) y el perfil de Blade no tiene binding de `DamageConfirmed`. El de `AtomicSlicImpact` (`ProjectileImpact` semántico) tampoco. Solo `MultiFinalImpact`, que entra por legacy, frena.
- Posible doble reacción por golpe. El impacto legacy dispara `EnemyHitFeedback` (`playEnemyReaction: true`, :89, :317-321) y el `DamageConfirmed` del mismo daño lo vuelve a disparar (:149-155). También se duplica `EnemyDeathFeedback.RecordHit`.
- `ReducedMotion` anula por completo el empuje y el squash del enemigo (`EnemyHitFeedback.cs:62`, :106). Los multiplicadores `_reducedMotionDisplacementScale` / `SquashScale` del perfil (`EnemyReactionProfile.cs:46-51`) quedan muertos.
- `CreateFlashShells` duplica también los renderers deshabilitados, y `ApplyFlash` enciende el shell sin mirar `source.enabled` (`EnemyHitFeedback.cs:172-185`, :237). En Chaser, el `MeshRenderer` placeholder oculto (`Chaser.prefab:399`) aparecería en cada golpe. Hay que verificarlo en play.
- `ReducedMotion` tiene dos dueños: `UserSettingsService` y `PresentationAccessibilityRuntime`. Los dos escriben las mismas opciones y `EnemyReactionRuntime.ReducedMotion` (`EnemyReactionProfile.cs:174-190`), y gana el último. `PauseMenuUI` sincroniza en un solo sentido (`PauseMenuUI.cs:334-344`).

- UI incompleta. `ImportantOnly` no se puede elegir (dropdown Off/Full, `PauseMenuUI.cs:548`). `HitStopEnabled` y `Quality` solo se cambian desde el sandbox (`WeaponSandboxDebugUI.cs:638-642`), así que producción queda fija en High con hit-stop on.
- `EnemyDeathReactionVfx`: `Statuses`, `Critical` y `WeaponType` se guardan en el pending y no se leen (`EnemyDeathReactionVfx.cs:42-46`, `:104-119`). Además hay allocations por muerte y por frame: `new List<int>` en `FlushPending` (`:127`) y `new Mesh + BakeMesh` (`:285-292`).
- `CombatTextDirector.Tick` mide GC y Stopwatch en cada frame, también en builds de release (`:214-254`). `ResolveBurnAnchor` hace `GetComponent<Collider>` / `GetComponentInChildren<Renderer>` por tally de burn y por frame (`:754-759`). `WorldAnchorHeight` = 4.34 en el asset contra 1.25 en el código, y `DistantScaleMultiplier` = 1 deja sin efecto a `FullSizeDistance`.

6 Cámara y aim

Cámara over-the-shoulder con mouse. El aim sale del centro de pantalla. La regla del sistema es separar la pose de **gameplay** de la de **presentación**: el orbit, la colisión y el rayo de aim usan la pose de gameplay. Shake y FOV kick se suman después, solo al transform visible, y nunca entran al aim (`ThirdPersonCamera.cs:293-294`). La retícula de pantalla queda fija en el centro. El aim assist mueve el punto de impacto real, no la retícula.

Scripts

Script	Rol
<code>Assets/Scripts/ThirdPersonCamera.cs</code>	Orbit yaw/pitch, hombro, colisión, impulsos de presentación, lock de cursor
<code>Assets/Scripts/Player/ReticleAimProvider.cs</code>	Rayo de gameplay, punto de impacto, aim assist, predicción de mortero
<code>Assets/Scripts/Player/AimSolution.cs</code>	<code>AimSolution</code> (origen → punto, frame) y <code>WeaponAimPolicy</code>
<code>Assets/Scripts/Weapon/Managers/WeaponManager.cs</code>	Resuelve el <code>AimSolution</code> por frame y lo pasa a las armas
<code>Assets/Scripts/Weapon/UI/ReticlePresentationLogi c.cs</code>	Modo de retícula por tipo de arma, progreso del lock de cohete
<code>Assets/Scripts/Weapon/UI/ReticleHud.cs</code>	Canvas de retícula, marcador de mortero en mundo, flash de weak point
<code>Assets/Scripts/GameFeel/CameraFeedbackController .cs</code>	Traduce cues de combate a impulsos de cámara

`ThirdPersonCamera` corre con `[DefaultExecutionOrder(-100)] (:5)`. Así el aim de gameplay ya está actualizado cuando corre el `Update` de `WeaponManager`.

Cámara: valores

Componente en la Main Camera de `GameplayScene` (`GameplayScene.unity:5407-5436`). FOV de la cámara: 90 (:5321).

Campo	Script	
<code>_pivotHeight</code>	1,6	1,5
<code>_shoulderOffset</code>	0,6	1
<code>_cameraDistance</code>	3,5	3,5
<code>_horizontalSensitivity</code> / <code>_verticalSensitivity</code>	0,12 / 0,12	0,12 / 0,12 (las pisa Settings)
<code>_minPitch</code> / <code>_maxPitch</code>	-55° / 65°	-70° / 70°
<code>_cameraCollisionMask</code>	Everything	Everything
<code>_cameraCollisionRadius</code> / <code>_cameraCollisionPadding</code>	0,25 / 0,12	igual
<code>_presentationImpulseDecay</code>	12	12
Tope de impulso: posición / rotación / FOV	0,35 / 5° / 5	igual
Reduced motion: posición / rotación / FOV	0,2 / 0,35 / 0	igual

`ApplyMainGameOrbitDefaults()` (:234-253) reescribe los valores de script (1,6 / 0,6 / -55.65). La llaman los builders del sandbox y de balance (`WeaponTestingSandboxManager.cs:581`, `BalanceTestingSceneBuilder.cs:135`). Por eso el sandbox no encuadra igual que la escena principal.

Cámara: frame

- En `Update` (:255-287): - Suma el delta de mouse a yaw y pitch, salvo con `_lookBlockedByUi`, y clampa el pitch. - `pivot = target + up × pivotHeight`. - `anchor = pivot + orbit × (hombro, alturaCam, 0)`. - `deseada = anchor - forward × distancia`. - Resuelve la colisión y guarda `_gameplayPosition` / `_gameplayRotation`.
- Colisión (:311-351): `SphereCast` desde el ancla hacia la posición deseada, ignorando el propio target y sus hijos. Deja la cámara en `hit - padding`, con mínimo 0,05 m.
- En `LateUpdate` (:289-303): - Aplica al transform `gameplay + impulsos × _cameraFeedbackScale`. - Aplica `FOV = base + kick`. - Decae los impulsos con `exp(-12 × unscaledDt)`. Los impulsos siguen en hit-stop.
- `GameplayCenterRay` (:78) = rayo desde la pose de gameplay más el near clip, en la dirección de gameplay. Es el rayo que usa el aim.

`AddPresentationImpulse` (:189-223) suma y clampa. Devuelve `false` si el shake está apagado, si la escala es 0 o si el impulso es nulo. Con `reducedMotion` escala primero por los factores de reduced motion.

`SetLookBlockedByUi` (:111-128) libera o bloquea el cursor. Lo usan `PauseMenuUI`, `LevelUpChoiceUI`, `CraftingUI`, `RunEndScreenUI` y las UIs de debug.

Aim de gameplay

`WeaponManager.GetAimDirection` (`WeaponManager.cs:355-363`) llama

`ReticleAimProvider.ResolveWeaponAim(spawn.position, armaManual)`. En `Update` (:66-81) lo resuelve una vez, evalúa la pose animada del esqueleto y lo vuelve a resolver con el muzzle actualizado. El aim se calcula antes del chequeo de `timeScale`, también en pausa.

Valores del provider (`player.prefab:831-837`, iguales al script): `_maxAimDistance` 150, `_aimMask` Everything, `_aimAssistRadius` 0,35 m. `_aimCamera` e `_ignoredRoot` vacíos: resuelven a `Camera.main` y al root del jugador.

Flujo de `TryGetAimSolution` (`ReticleAimProvider.cs:50-56`):

- Rayo.** `GameplayCenterRay` si la cámara tiene `ThirdPersonCamera` activa. Si no, el viewport en (0,5; 0,5) (:65-74).
- Raycast.** Hasta 150 m, sin triggers. Ignora el cuerpo del jugador y se queda con el hit más cercano (:141-160).
- Sin hit.** El punto es la intersección del rayo con una esfera de radio = alcance del arma (`BaseRange`), centrada en el muzzle (:247-265). Así un disparo al vacío converge con la retícula a la distancia del arma, no a 150 m.
- Aim assist.** Solo si la política del arma lo pide y el rayo no tocó un enemigo directo (:163-164). `SphereCast` de 0,35 m. Por candidato busca el punto del collider más cercano al rayo (ternaria de 24 pasos) y descarta los tapados por otra geometría (:168-222). Gana el menor gap. No hay target persistente entre frames.
- `AimSolution` = origen del muzzle → punto, con `Direction` normalizada y `FrameNumber` (`AimSolution.cs:4-21`).

`WeaponAimPolicy.PreferDamageableAimPoint` (`AimSolution.cs:25-31`):

Arma	Aim assist
RocketLauncher	Sí
AutomaticCannon	Sí, salvo con Path B avanzado
Resto	No

Un collider cuenta como "enemigo" si tiene `EnemyRegistryMember` o `IDamageable` en los padres (:234-245).

Predicción de mortero

`TryGetMortarTerrainImpact` (:83-114) recorre la parábola de `MortarTrajectory` en 64 cuerdas con `SphereCast` del radio del proyectil. Ignora jugador, `IDamageable` y miembros del registro de enemigos: solo cuenta terreno (:116-131).

Retícula

`ReticlePresentationLogic.ResolveMode` (`ReticlePresentationLogic.cs:25-37`):

Arma	Modo
Flamethrower, RotatingBlade	<code>WideBrackets</code> (250×70 px)
AutomaticCannon, RocketLauncher	<code>CircleDot</code> (Ø 34, punto 5)
RocketLauncher cargando lock	<code>RocketLock</code> : marco de 180×100 a 1344×756 que se agranda con el progreso
Mortar	<code>Mortar</code> : V en pantalla y anillos en el mundo
Sin arma manual	<code>Hidden</code>

Valores en `player.prefab:852-877`, iguales al script.

`ReticleHud.LateUpdate` (`ReticleHud.cs:115-154`):

1. Lee el arma manual de `WeaponManager` o, si existe, del sandbox.
2. Resuelve el modo y actualiza el marco de cohete.
3. En modo mortero, recalcula la predicción cada 0,04 s (`_mortarPredictionInterval`) con el `AimSolution` del frame. Si el aim no es de este frame, oculta el marcador (:474-495).
4. Flash rojo de 0,18 s en toda la retícula ante `WeaponWeakPointFeedback.WeakPointHit`.

La retícula queda anclada al centro del canvas (:137-138).

Deuda / puntos abiertos

- `_minimumDistanceFromLookPoint` (0,65) está serializado pero ningún código lo lee (`ThirdPersonCamera.cs:48`, solo aparece en :248). La cámara puede quedar a 0,05 m del ancla contra una pared.
- Los factores de reduced motion de la cámara casi nunca se aplican. `UserSettingsApplier` apaga todo el shake con reduced motion activo (`UserSettingsApplier.cs:239`) y `AddPresentationImpulse` sale antes de escalar (:202-203). Solo aplican cuando el reduced motion de accesibilidad (`SaveManager`) difiere del de Settings. Ver 13-Settings-y-accesibilidad.md.
- `ReticlePresentationLogic.TryProjectAimPoint` no tiene llamadores. Con aim assist, la bala va a un punto distinto del centro y la retícula no lo muestra.
- El look lee `Mouse.current.delta` directo (:260-268). No hay gamepad ni suavizado.
- `ApplyMainGameOrbitDefaults` contradice los valores de `GameplayScene` (pivote, hombro, pitch). Las escenas de test encuadran distinto y el aim assist se siente distinto con hombro 0,6 que con hombro 1.
- El aim se resuelve dos veces por frame cuando hay animation driver (`WeaponManager.cs:66-79`): son dos raycasts más el `SphereCast` de assist.

7 UI y HUD

Toda la UI del juego es uGUI (`com.unity.ugui 2.0.0`) con texto `TextMeshPro (com.unity.textmeshpro 3.2.0)` y navegación por `InputSystemUIInputModule (com.unity.inputsystem 1.19.0)`. No hay UI Toolkit en runtime: no hay `.uxml`, `.uss` ni `UIDocument`. El criterio actual es "vista autorada en escena o prefab, presenter que solo bindea": los presenters no construyen jerarquía en runtime. Los builders de jerarquía existen, pero son editor-only (`#if UNITY_EDITOR`, métodos `AuthorUi`). El indicador trasero es un mesh en mundo y no usa Canvas: ver `44-Rear-threat.md`.

Canvas

Todos los canvas raíz son `Screen Space Overlay` con `CanvasScaler` en `Scale With Screen Size`, `1920×1080`, `match 0,5`. Orden en `GameplayScene` (valores serializados):

Canvas	Sorting	Origen
<code>PauseRoot</code> (canvas anidado, override sorting)	32760	Agregado en escena sobre la instancia HUD V2 (<code>GameplayScene.unity:31240</code>)
<code>CraftingMenu</code>	5100	<code>Assets/Prefabs/UI/RunMenus/CraftingMenu.prefab:3157</code>
<code>LevelUpMenu / WeaponSelectionMenu</code>	5000	<code>Prefabs RunMenus (:1937 / :2135)</code>
<code>LevelUpStatFeedbackCanvas</code>	4900	Escena (:2045)
<code>AchievementUnlockToastCanvas</code>	750	Escena (:32930)
<code>ReticleHUD_Canvas</code>	650	Escena (:17621); default de <code>ReticleHud._sortingOrder</code>
<code>GameplayHudCanvas</code>	600	<code>GameplayHud V2.prefab</code> ; lo fuerza <code>GameplayHudRoot._sortingOrder</code> en <code>Awake</code>
<code>SurvivorHUD_Canvas</code>	500	Legacy, dentro de un <code>GameObject</code> inactivo
<code>GameplayUI_Canvas</code>	400	Legacy (<code>UIManager</code>), dentro de un <code>GameObject</code> inactivo
<code>MaterialInventoryHUDCanvas</code>	100	Escena (:4386)

Todo cuelga de la raíz `UI (GameplayScene.unity:30161)`, igual que el `EventSystem (:23511)`. En `TitleScreen`: `MainMenuCanvas (1000)` y `FeedbackCanvas (1500)`. `_includeTestingButtons` está en 0 (`TitleScreen.unity:19819`).

HUD de gameplay: prefabs

Prefab	Estado
<code>Assets/Prefabs/UI/GameplayHud V2.prefab</code>	El que usan <code>GameplayScene (GameplayScene.unity:9730)</code> y <code>SampleScene</code> . Suma <code>LevelExitHud</code> y 2 <code>Image</code> respecto de <code>V1</code>
<code>Assets/Prefabs/UI/GameplayHud .prefab</code>	Sin referencias en escenas ni prefabs. Es el destino de <code>GameplayHudPrefabBuilder</code>
<code>Assets/Prefabs/UI/PlayerBarsHUD.prefab</code>	Sin referencias por GUID. Solo lo lee el builder por path (<code>GameplayHudPrefabBuilder.cs:17</code>)
<code>Assets/Prefabs/UI/RunMenus/*.prefab</code>	<code>LevelUp</code> (3 cartas), <code>WeaponSelection</code> (2 cartas), <code>Crafting</code> (3 slots). Los instancia la escena bajo <code>UI</code>

Contenido de V2, bajo `GameplayHudCanvas`: `PlayerCombatFeedback`, `BossHealthBarHud`, `OverheatObjectiveHud`, `OffscreenObjectiveIndicators`, `BottomStrip` (`ColumnLeft` con barras, `ColumnCenter` con pasivos, `ColumnRight` con armas), `PauseMenuUI`, `RunEndScreenUI` (con `RunEndRoot`) y `LevelExitHud`.

La jerarquía visual de la pausa (`PauseRoot` con 8 hijos) no está en el prefab. Es un objeto agregado en escena sobre la instancia (`GameplayScene.unity:9727-9728`), con `_root` de `PauseMenuUI` sobrescrito (:9053-9055). `SampleScene` tiene sus propias copias.

Scripts

Script	Rol
<code>Assets/Scripts/Weapon/UI/GameplayHudRoot.cs</code>	Raíz del HUD V2. Solo fija el sorting (600) del canvas hijo
<code>Assets/Scripts/Weapon/UI/GameplayHudHierarchyBuilder.cs</code>	Editor-only. Arma canvas, capas y <code>BottomStrip</code> , y la jerarquía de <code>RunEnd</code>
<code>Assets/Scripts/Weapon/Editor/GameplayHudPrefabBuilder.cs</code>	Menús <i>Build GameplayHud Prefab</i> (:21), <i>Rebuild BottomStrip In Prefab</i> (:49), <i>Wire SampleScene GameplayHud</i> (:259). Escriben <code>GameplayHud.prefab</code> (:15)
<code>Assets/Scripts/Editor/GameplayUiSceneMigration.cs</code>	<i>Organize Current Scene Under UI / Migrate All Player Scenes Under UI</i> . Mueve canvases y <code>EventSystem</code> bajo <code>UI</code> , cablea menús <code>RunMenus</code> y llama <code>AuthorUi</code> de cada presenter (:82-148)
<code>Weapon/UI/HudUiFactory.cs</code> , <code>HudUiWire.cs</code> , <code>HudBottomStripLayouts.cs</code> , <code>TmpUiHelper.cs</code>	Helpers de construcción y búsqueda por nombre
<code>Weapon/UI/PlayerBarsHud.cs</code>	Esfera = heat/overheat, barra superior = HP, inferior = XP. Por eventos
<code>Weapon/UI/PassiveLoadoutHud.cs</code>	6 slots de pasivos. <code>PassiveItemManager.OnInventoryChanged</code>
<code>Weapon/UI/WeaponClusterHud.cs</code>	3 slots de arma, munición, cooldown de rotación, habilidad y cargas de dash
<code>Weapon/UI/ReticleHud.cs</code> (+ <code>ReticlePresentationLogic.cs</code>)	Retrícula del arma manual, marcador de mortero, flash de punto débil
<code>Weapon/UI/BossHealthBarHud.cs</code> , <code>OverheatObjectiveHud.cs</code> , <code>OffscreenObjectiveIndicators.cs</code>	Jefe, objetivo de overheat, flechas fuera de pantalla hacia jefe/élite
<code>Weapon/UI/PlayerCombatFeedback.cs</code>	Flash de daño y overlays de invulnerabilidad, stun y quemadura
<code>Weapon/UI/PauseMenuUI.cs</code>	Pausa, ajustes in-run y stats. Tabs de <code>StatAttributionPanel</code> solo en editor/dev (:58, :80)
<code>Weapon/UI/StatAttributionPanel.cs</code> , <code>LastHitRecorder.cs</code>	Panel dev: fuente de cada stat y último golpe
<code>Weapon/UI/RunEndScreenUI.cs</code>	Pantalla de victoria o derrota. Singleton Instance
<code>Weapon/UI/DebugUI.cs</code>	Overlay dev (F3): FPS, heat y log. P pausa y numpad ± cambia el time scale
<code>Weapon/UI/UiManager.cs</code> , <code>SurvivorHud.cs</code>	Legacy. Ver abajo
<code>Assets/Scripts/Level/LevelExitHud.cs</code>	Llaves, carga de puerta y presión de salida
<code>Assets/Scripts/XP/LevelUpChoiceUI.cs</code>	Controller de cartas de level-up y de primera arma. Vive en <code>player.prefab</code>
<code>Assets/Scripts/Economy/CraftingUI.cs</code>	Controller del menú de crafting. Vive en <code>player.prefab</code>

Script	Rol
Assets/Scripts/UI/RunMenus/*	ChoiceMenuView / ChoiceCardView (cartas), CraftingMenuView / CraftingWeaponSlotView, RunMenuContent (SO con copy opcional por arma)
Assets/Scripts/UI/WeaponUiIconCatalog.cs, AchievementUiIconCatalog.cs	SO en Resources/UI/..., con carga lazy y setter para tests
Assets/Scripts/TitleScreenController.cs	Botones del título, sección DEV y ruteo a escenas
Assets/Scripts/UI/MainMenu/*	TitleScreenScreenStack, SettingsScreenUI, MainMenuPresentationController (+Profile, ItemView, audio, punch, fondo), ScrapSceneTransition, vistas de objetivos y unlocks
Assets/Scripts/GameplayPause.cs	Contador estático de locks de pausa de UI

Legacy: UIManager y SurvivorHud

En `GameplayScene` los dos están apagados por partida doble. `UIManager` tiene el componente en `m_Enabled: 0` (`GameplayScene.unity:24734`) y su `GameObject UIManager` inactivo. `SurvivorHud` también tiene `m_Enabled: 0` (`:22780`) y su `GameObject StatHud` inactivo. Sus canvases (400 y 500) siguen serializados.

Referencias vivas: `GameplayUiSceneMigration.cs:113-122` les llama `AuthorUi` en cada migración; `GameplayHudPrefabBuilder.cs:307` desactiva `SurvivorHud`; `SupportUiAuthoringTests.cs:17-18` los testea. En runtime no los referencia nada.

Cómo pausan los menús

`GameplayPause` (`Assets/Scripts/GameplayPause.cs`) es solo un contador (`Push/Pop/SetHeld`), con reset en `SubsystemRegistration`. No toca `Time.timeScale`. Lo consultan `PlayerMovement.cs:314`, `HeatManager.cs:290` y `WeaponPresentationController.cs:448`. Cada menú pone `Time.timeScale = 0`, guarda su propio valor previo y toma el lock:

Quién	Pausa	Restaura
PauseMenuUI	<code>SetPauseState(true, 0)</code> (<code>:248-267</code>)	Valor guardado si <code>> 0,001</code> , si no 1
LevelUpChoiceUI	<code>:112-114</code> , si <code>_pauseWhileChoosing</code> (1 en <code>player.prefab:917</code>)	Previo si <code>> 0</code> , si no 1 (<code>:143</code>)
CraftingUI	<code>:107-109</code>	Previo si <code>> 0</code> , si no 1 (<code>:124</code>)
<code>GameManager.EnterEndState</code>	<code>:101-102</code>	Solo al recargar (<code>ResetTimeScaleForReload</code> , <code>:118-122</code>)
HitStopController	<code>timeScale = 0</code> sin lock	Su propio valor guardado
DebugUI	P con el panel visible, sin lock (<code>:213-224</code>)	Su propio valor guardado

Todos los menús llaman `ThirdPersonCamera.SetLookBlockedByUi`, que desbloquea y muestra el cursor.

Flujo: pausa

1. `PauseMenuUI.Update` (`:114`) detecta `Escape` (`Input System`). Si `_root` es nulo, no hace nada.
2. `CanPause` (`:164-182`) vuelve a resolver refs. Rechaza si hay level-up o crafting visibles (y deja el cursor libre) o si `GameManager` no está en `Playing`.
3. `ShowPause`: guarda el time scale, `SetPauseState(true, 0)` (activa `PauseRoot`, cursor libre, cámara bloqueada, lock), sincroniza ajustes y stats, y foco en `Resume`.

4. Mientras está abierto, `RefreshRunStats` corre cada frame.
5. Escape cierra en orden: primero la tab dev de atribución, después el panel de ajustes, y recién entonces hace Resume (:216-230).
6. `Quit` → `ReturnToTitle`: despausa y va a `SceneNavigation.LoadScene()`. Si falla, vuelve a pausar.

Flujo: carta de level-up

1. `PlayerXP.OnLevelUp` → `LevelUpOrchestrator.HandleLevelUp` encola el nivel. Los niveles se procesan uno por uno (`LevelUpOrchestrator.cs:43-61`).
2. `PassiveItemLevelUpHandler.PresentAndApplyCoroutine` filtra el pool por desbloques, arma entre 2 y 3 ofertas (`_choicesOffered = 3` en `player.prefab:947`) y llama `LevelUpChoiceUI.PresentCoroutine`.
3. `ShowChoice` (:87-116) valida que la vista tenga cartas suficientes (`CanPresent`), bindea las cartas (`ChoiceMenuView.Show`: nombre, resumen/descripción de `RunMenuContent` si es arma, icono), bloquea la cámara y pausa.
4. Click en una carta → `CompleteChoice` → `Hide` (restaura time scale y suelta el lock) → callback con el índice.
5. El handler aplica el pasivo. Después, `PlayerStatsLevelUpHandler` aplica las stats automáticas y `LevelUpStatFeedback.Show` las muestra (canvas 4900).
6. Si el controller se desactiva con la carta abierta, se completa con `-1` (:148-154).

La elección de primera arma usa el mismo controller con `WeaponSelectionMenu` (`RunStartWeaponChoice.cs:53`).

Flujo: fin de run

1. `PlayerHealth.OnPlayerDied` → `GameManager.OnPlayerDied`, O `TriggerVictory` desde la puerta de salida.
2. `EnterEndState` (:99-115): estado, `timeScale = 0`, lock, `ReportRunToSaveSystem`. Después, `RunEndScreenUI.Instance.Show`, o error si no hay pantalla autorada.
3. `Show` (:43): título y color según victoria/derrota; stats (tiempo, kills, nivel, jefes); cámara bloqueada; `RunEndRoot` activo.
4. `Retry` (:134-143): `ResetTimeScaleForReload` y `SceneManager.LoadScene(buildIndex actual)`. *Main menu*: `SceneNavigation.LoadScene()`.

Título

`TitleScreenController` cablea Play, Objectives, Settings, Quit (oculto en WebGL) y la sección DEV (`_developerRoot`, apagada salvo `_includeTestingButtons`). Arma la navegación explícita vertical (:276-299). Play usa `ScrapSceneTransition.TryLoad` (persistente, `DontDestroyOnLoad`, `ScrapSceneTransition.cs:164`) y, si falla, carga directo.

`TitleScreenScreenStack` maneja una sola pantalla local por vez (Objectives, Settings, QuitConfirmation): animación con `unscaledDeltaTime` (0,22 s de apertura / 0,18 s de cierre, o lo que diga el profile), cancel por `ICancelHandler` y `InputActionReference`, foco de retorno. `SettingsScreenUI` tiene tres categorías (Controls, Audio, Feedback), reset con confirmación (armado de dos pasos) y escribe `UserSettingsService`.

Tests (Assets/Tests/Editor)

`GameplayUiSceneTests` (6 escenas: una raíz UI, menús y retícula autorados bajo ella, ningún canvas de pantalla fuera de UI), `GameplayHudAuthoringTests`, `SupportUiAuthoringTests`, `PauseMenuUITests` (navegación, Escape, sorting sobre DebugUI, bloqueos modales, ajustes), `GameplayPauseTests`, `RunMenuUITests` (cartas 3/2, crafting 3 slots/6 materiales, sin crear UI en runtime), `ReticleHud*Tests`, `TitleScreenControllerTests`, `TitleScreenScreenStackTests`, `ScrapSceneTransitionTests`, `ObjectivesMenuUIPresenterTests`, `AchievementToastAuthoringTests`, `*UiIconCatalogTests`, `StatAttributionTests`, `SceneNavigationTests`, `UserSettingsServiceTests`.

Deuda / puntos abiertos

- `GameplayHudPrefabBuilder` construye y reconstruye `GameplayHud.prefab` (`GameplayHudPrefabBuilder.cs:15`), que no está en ninguna escena. Las escenas usan `GameplayHud V2.prefab`. El warning de `GameplayHudRoot.cs:20` manda a ese menú. Hay que decidir si se borra V1 y se apunta el builder a V2, o si se borra el builder.
- La pausa no está en el prefab. `PauseRoot` y sus hijos son objetos agregados en cada escena sobre la instancia V2 (`GameplayScene.unity:9727-9728`; `SampleScene` tiene dos `PauseRoot`). Una instancia nueva de V2 queda con `_root` nulo, y `PauseMenuUI.Update` sale en silencio (:116). Hay que aplicar el override al prefab.
- Hay dos almacenes de accesibilidad. El título escribe `UserSettingsService.ScreenShake/ScreenFlash` (`SettingsScreenUI.cs:278-279`). La pausa escribe `ReducedShake/ReducedFlash` en `SaveManager.PresentationAccessibility` (`PauseMenuUI.cs:546-547`). Solo `ReducedMotion` se sincroniza (:334-344, :539-544). Temblor y flash quedan con valores distintos según desde dónde se cambien.
- El time scale no tiene dueño: cada menú guarda su propio previo. `DebugUI` está activo en `GameplayScene` (:11587), no tiene gate de dev build (F3 funciona en release, `DebugUI.cs:99-120`) y su P pone el time scale sin tomar el lock. Con una carta de level-up o el crafting abiertos, lo vuelve a 1 y el mundo corre debajo del menú, mientras `GameplayPause` sigue bloqueando solo al jugador.
- `LevelUpChoiceUI.PresentCoroutine` ignora `title` (:51-56). `PresentWeaponSelectionCoroutine` pasa null como título. Los títulos en castellano de los llamadores (`PassiveItemLevelUpHandler.cs:60`, `WeaponLevelUpHandler.cs:77`) y el de `RunStartWeaponChoice.cs:53` nunca se muestran. `Show` hardcodea textos en inglés (:84).
- `UIManager` y `SurvivorHud` están muertos en runtime, pero la migración les sigue llamando `AuthorUI`, y sus canvases (400 y 500) siguen en la escena. Son candidatos a borrarse junto con `SupportUiAuthoringTests:17-18`.
- `GameplayHudRoot._playerBarsContent` (:8-9) no se lee nunca. Su tooltip promete una jerarquía placeholder en `Awake` que no existe.
- Desde `gameplay` no se usa `ScrapSceneTransition:Retry` carga con `SceneManager.LoadScene` directo (`RunEndScreenUI.cs:142`), y `Quit` y `Main menu` usan `SceneNavigation.LoadTitle()`. La transición solo existe en el camino título → juego. Además, el crafting solo se cierra con su botón: `Escape` queda bloqueado por `CanPause` (:169-177) y `CraftingUI` no maneja cancel.

8 Audio

Audio sin `AudioMixer`: no hay assets `.mixer` en el proyecto y todas las `AudioSource` de `TitleScreen` y `GameplayScene` tienen `OutputAudioMixerGroup` vacío. El volumen se controla multiplicando en código. Cada escena tiene su propio `AudioManager` (no persiste) con una playlist de BGM y un canal de SFX genérico. El audio de armas tiene su propio pool de voces 3D dentro de la presentación de armas, y el menú y la transición reproducen sus clips por su cuenta.

Scripts

Script	Rol
<code>Assets/Scripts/Audio/AudioManager.cs</code>	Singleton por escena: SFX genéricos, playlist de BGM, capa de Overheat
<code>Assets/Scripts/Audio/BgmTrackSelector.cs</code>	Elección de pista: <code>AlternateTwo</code> o <code>ShuffleBag</code>
<code>Assets/Scripts/GameFeel/WeaponAudioDirector.cs</code> + <code>Weapon/Presentation/WeaponAudioVoicePool.cs</code>	SFX de armas por cue, con voces 3D
<code>Assets/Scripts/UI/MainMenu/MenuAudioFeedback.cs</code>	Navegación y confirmación del menú de título
<code>Assets/Scripts/UI/MainMenu/ScrapSceneTransition.cs</code>	Clips de aviso, impacto y reveal de la cortina
<code>Assets/Scripts/Settings/UserSettingsApplier.cs</code>	Empuja los volúmenes SFX y música de settings al <code>AudioManager</code>

AudioManager

`[DefaultExecutionOrder(-60)]`. Instance se asigna en `OnEnable` y se limpia en `OnDisable`, sin chequeo de duplicados (`AudioManager.cs:56-66`). Emite `BecameAvailable` para que `UserSettingsApplier` le aplique los volúmenes.

Tres fuentes:

Fuente	Uso
<code>_sfx</code>	<code>PlayOneShot(clip, _sfxVolumeScale)</code> (:278-284)
<code>_musicNormal</code>	Playlist, sin loop. El volumen es el de música
<code>_musicOverheatLayer</code>	Loop opcional. Volumen 0 fuera de Overheat

En `Start` se suscribe a `PlayerXP.OnLevelUp` y a `OverheatManager.OnOverheatStarted/Finished` (búsqueda por `FindAnyObjectByType`, :81-93). Después arranca la playlist y la capa de Overheat en idle (:68-73).

Helpers estáticos: `TryPlayShoot`, `TryPlayEnemyHit`, `TryPlayEnemyDeath` y `TryPlayPlayerHurt` (:287-293). `Hurt` tiene un cooldown de 1 s en unscaled time, así los golpes seguidos no apilan el SFX (:244-251).

Valores por escena

Campo	Script		
<code>_sfx</code>	—	asignado	vacío
<code>_musicOverheatLayer</code>	—	asignado	vacío
<code>_shoot</code>	—	<code>shoot.wav</code>	—
<code>_enemyHit</code> / <code>_enemyDeath</code>	—	vacíos	—

Campo	Script		
<code>_levelUp</code>	—	<code>level_up.wav</code>	—
<code>_overheatStart</code> / <code>_overheatEnd</code>	—	<code>010.wav</code> los dos	—
<code>_playerHurt</code>	—	<code>player_hurt.wav</code>	—
<code>_bgmTracks</code>	—	Gameplay v1–v4 (.ogg)	Title Screen v1–v2 (.ogg)
<code>_bgmMode</code>	ShuffleBag	ShuffleBag (1)	AlternateTwo (0)
<code>_bgmOverheatLayer</code>	—	vacío	vacío
Pausa entre pistas	10–30 s	10–30 s	10–30 s
<code>_sfxVolumeScale</code> / <code>_musicMainVolume</code>	0,2 / 0,2	0,2 / 0,2	0,2 / 0,2
<code>_musicOverheatVolume</code>	0,1556	0,1556	0,1556

Las `AudioSource` de la escena quedan en volumen 1, sin loop, prioridad 128. La de música no tiene `PlayOnAwake`.

Playlist

1. `StartMainBgm` (:124-138): si hay al menos una pista no nula, pone `loop = false` y el volumen de música, y crea el selector. `AlternateTwo` con un número de pistas distinto de 2 cae a `ShuffleBag` (:140-147).
2. `RunPlaylist` (:149-180): - Pide un índice. Si el clip es nulo, reintenta hasta N veces. - Hace `Play` y espera `!isPlaying` frame a frame. - Espera un silencio aleatorio entre min y max con `WaitForSecondsRealtime`. - Repite.
3. `BgmTrackSelector` (`BgmTrackSelector.cs:36-88`): - **AlternateTwo**. Arranca con una pista al azar y después alterna 0/1. - **ShuffleBag**. Fisher-Yates. Al rellenar la bolsa, si la primera es la última que sonó, la intercambia con otra, así nunca repite dos seguidas entre bolsas. - Con una sola pista, siempre la 0.

Capa Overheat

`SetOverheatLayerActive` (`AudioManager.cs:217-230`) sube la capa a:

```
clamp01(0,1556 × volumenMúsica / 0,2)
```

Queda proporcional al slider de música, con referencia en el default 0,2 (:272-276). Cambiar el volumen de música reescala la capa solo si está sonando (:259-270). En `GameplayScene` no hay clip de capa, así que al entrar en Overheat solo suena el SFX `010.wav`.

Volumen: quién aplica qué

Emisor	Factor de volumen	Fuente del valor
AudioManager SFX	<code>_sfxVolumeScale</code>	Settings, vía applier
AudioManager música	<code>_musicMainVolume</code> en <code>AudioSource.volume</code>	Settings, vía applier
Armas (<code>WeaponAudioVoicePool</code>)	<code>cue.Volume × SfxVolume × intensidad × capa</code>	<code>AudioManager.SfxVolume</code> , o 1 si no hay manager (<code>WeaponPresentationController.cs:616-619</code>)
Menú de título	<code>volumenAuthorred × SfxVolume</code> (navegación 0,55, impacto 0,9)	<code>UserSettingsService</code> directo (<code>MenuAudioFeedback.cs:71-75</code>)
Transición	<code>0,65 / 0,95 / 0,7 × SfxVolume</code>	<code>UserSettingsService</code> directo

Voces de armas: 16 voces con spatial blend 1 (`player.prefab:1041-1042`, iguales al script).

Assets

Assets/Audio:

- **BGM.** Seis `.ogg`: Gameplay v1–v4 y Title Screen v1–v2. Están en Streaming y Vorbis (`loadType: 2`). Cada uno tiene un `.m4a` duplicado sin referencias.
- **SFX.** `.wav` en Decompress On Load: `shoot`, `level_up`, `player_hurt`, `010`, `UI_0..2` y `click` (reject del menú).
- `Junkyard-Pulse-Loop.ogg` solo lo referencia `SampleScene`, que no está en el build.

Deuda / puntos abiertos

- **El SFX de disparo no suena nunca.** El único llamador de `TryPlayShoot` es `PlayerAutoAttack` (`PlayerAutoAttack.cs:64`), deshabilitado en el prefab (`player.prefab:619`). El clip `shoot.wav` está asignado sin uso. Las armas reales suenan por su propio pool.
- `_enemyHit` y `_enemyDeath` están vacíos en `GameplayScene`. Las llamadas de `EnemyHealth` (`EnemyHealth.cs:159,167`) no hacen nada. Hay que confirmar si el audio de impacto de armas lo cubre a propósito.
- `_bgmOverheatLayer` está vacío. La capa de Overheat y su `AudioSource` están cableadas, pero en la run no suena nada.
- Overheat start y end usan el mismo clip (`010.wav`), que también es el `_localOpenClip` del menú.
- Sin mixer: no hay ducking, ni bus de SFX o UI, ni snapshots de pausa. La música y los loops siguen sonando con `timeScale = 0`: nada llama `AudioListener.pause` ni pausa las fuentes.
- Tres caminos leen el volumen SFX de fuentes distintas (`AudioManager`, `UserSettingsService` directo, fallback 1). En `GameplayScene` arrancada desde el editor, sin servicio de settings, el menú no aplica y las armas usan el 0,2 serializado del `AudioManager`.
- `Instance` es el último `AudioManager` habilitado, sin guarda de duplicados (`AudioManager.cs:56-60`). Si se deshabilita uno, `Instance` queda nulo aunque haya otro activo.
- Seis `.m4a` duplicados de la BGM sin referencias en `Assets/Audio/BGM`.
- 10 a 30 s de silencio entre pistas (`_bgmGapMinSeconds/_bgmGapMaxSeconds`) en las dos escenas. En una run corta es mucho tiempo sin música; conviene confirmarlo con diseño.

PARTE 3

Infraestructura

9 Escenas y navegación

Carga de escenas sin additive ni async: todo es `SceneManager.LoadScene` síncrono por build index. `SceneNavigation` es la única tabla de destinos. `ScrapSceneTransition` pone una cortina animada encima y sobrevive a los cambios de escena, pero solo el menú de título la usa. La pausa mezcla dos mecanismos: `Time.timeScale` y un contador de locks de UI (`GameplayPause`).

Scripts

Script	Rol
<code>Assets/Scripts/SceneNavigation.cs</code>	Enum <code>SceneDestination</code> , nombres y paths, <code>Load</code> , <code>QuitApplication</code>
<code>Assets/Scripts/UI/MainMenu/ScrapSceneTransition.cs</code>	Overlay de transición persistente (singleton, <code>DontDestroyOnLoad</code>)
<code>Assets/Scripts/TitleScreenController.cs</code>	Botones del título, destinos DEV, foco, bloqueo de input durante la transición
<code>Assets/Scripts/GameplayPause.cs</code>	Contador estático de locks de pausa de UI
<code>Assets/Scripts/Weapon/UI/PauseMenuUI.cs</code>	Menú de pausa (Escape), settings in-game, volver al título
<code>Assets/Scripts/Weapon/UI/RunEndScreenUI.cs</code>	Pantalla de fin de run: Retry y menú principal
<code>Assets/Scripts/GameManager.cs</code>	Estado de la run. Congela el tiempo al terminar
<code>Assets/Prefabs/UI/MainMenu/ScrapSceneTransition.prefab</code>	Overlay authored

Build settings

`ProjectSettings/EditorBuildSettings.asset`:

Índice	Escena	En build	Destino en
0	<code>Assets/Scenes/TitleScreen.unity</code>	Sí	Title
1	<code>Assets/Scenes/GameplayScene.unity</code>	Sí	Play
—	<code>Assets/Scenes/SampleScene.unity</code>	No	—
2	<code>Assets/Scenes/Testing/WeaponTestingSandbox.unity</code>	Sí	WeaponSandbox
3	<code>Assets/Scenes/Testing/enemiesTesting.unity</code>	Sí	EnemiesTesting
4	<code>Assets/Scenes/Testing/test_balance.unity</code>	Sí	ninguno

`SceneNavigation.Load` (`SceneNavigation.cs:37-51`) resuelve el path fijo (`:84-94`) a build index con `SceneUtility.GetBuildIndexByScenePath`. Si la escena no está habilitada en Build Settings, loguea un error y devuelve `false` sin cargar. Antes de cargar llama `PrepareForSceneChange`, que solo pone `Time.timeScale = 1` (`:32-35`). `QuitApplication` sale de Play Mode en editor y llama `Application.Quit()` en build (`:73-82`).

Raíz persistente

`TitleScreen` tiene un root `PersistentSystemsRoot` (`TitleScreen.unity:11293`) con `UserSettingsService`, `UserSettingsApplier` y, como hijo, la instancia de `ScrapSceneTransition.prefab` (`:10117`).

- `UserSettingsService.Awake` hace `DontDestroyOnLoad` de su `GameObject`, que es el root.

- `ScrapSceneTransition.Awake` hace `DontDestroyOnLoad(transform.root)` (`ScrapSceneTransition.cs:163-164`).
- Al volver al título se carga otra copia del root. `UserSettingsService.Awake` (orden -220) ve que ya hay instancia y desactiva y destruye la copia entera, transición incluida (`UserSettingsService.cs:82-97`). Si igual quedara otra transición viva, su `Awake` loguea el duplicado y la oculta (`ScrapSceneTransition.cs:147-156`).
- `TitleScreenController.Awake` reemplaza sus referencias authored por las instancias vivas (`TitleScreenController.cs:54-57`).

`GameplayScene` no tiene servicio de settings ni transición. Si se arranca directo en el editor no hay cortina ni settings aplicados, y `UserSettingsApplier` no existe.

Transición

Timings en unscaled time (prefab `ScrapSceneTransition.prefab:313-318`, iguales al script):

Fase	Duración	Reduced motion
Warning (cuchilla de aviso)	0,1 s	Se saltea
Covering (placas)	0,38 s, curva con overshoot 1,035	Fade de alpha de 0,1 s con las placas ya cerradas
WaitingForScene	≥ 0,08 s tras <code>sceneLoaded</code> . Timeout 15 s	Igual
Revealing	0,3 s	Fade de 0,1 s

Clips: `UI_1.wav` (aviso), `UI_2.wav` (impacto) y `UI_0.wav` (reveal) (`ScrapSceneTransition.prefab:307-309`), escalados por el volumen SFX de settings. Con reduced motion, el impacto baja de 14 a 4 partículas.

Flujo (`ScrapSceneTransition.cs:195-420`):

1. `TryLoad(destino)`. Rechaza si ya hay una transición en curso, si el componente está inactivo o si faltan referencias authored. Guarda el nombre esperado y emite `TransitioningChanged(true)`.
2. `Update` avanza la máquina de estados con `unscaledDeltaTime`. Puede cruzar varias fases en un frame, con guarda de 8 iteraciones (:254-316).
3. Al cubrir del todo, invoca `SceneNavigation.Load` una sola vez (:369-405). Si falla o tira excepción, destapa la escena actual.
4. `HandleSceneLoaded` marca la carga solo si el nombre coincide con el esperado (:430-439). Sin ese evento, a los 15 s loguea un error y destapa igual (:340-367).
5. `Reveal` y `TransitioningChanged(false)`. El título bloquea input de menú y del screen stack mientras dura (`TitleScreenController.cs:206-210`).

Título

`TitleScreenController`:

- **Botones de producción.** Play, Objetivos, Settings y Salir, este último con confirmación. En WebGL, Salir se oculta (:355-362).
- **Destinos DEV.** Weapon Sandbox y Enemies Testing, detrás de `_includeTestingButtons`, que en la escena está en 0 (`TitleScreen.unity:19819`). Se activan con `SetTestingButtonsVisible`.
- **Carga.** `RequestSceneLoad` intenta la transición. Si no hay transición, o no arrancó y no hay otra en curso, loguea un error y carga directo (:159-171).
- **Navegación.** Arma navegación explícita vertical solo con los botones visibles e interactivables (:276-297). El foco inicial va a Play.
- **Preferencias.** Aplica reduced motion, shake y flash de `UserSettingsService` a la presentación del menú y al screen stack (:212-221).
- **Cursor.** Lo libera en `Awake`.

Pausa

`GameplayPause` (`GameplayPause.cs:7-48`) es un contador. `SetHeld(ref bool, bool)` garantiza un push/pop por dueño. Se resetea en `SubsystemRegistration`, no por escena.

Dueño del lock	Cuándo	
<code>PauseMenuUI</code>	Escape en run activa (<code>PauseMenuUI.cs:114-134</code>)	0. Al reanudar, restaura el valor guardado
<code>LevelUpChoiceUI</code>	Elección de mejora	0. Restaura el anterior
<code>CraftingUI</code>	Crafteo	0. Restaura el anterior
<code>GameManager</code>	Victoria o derrota (<code>GameManager.cs:98-103</code>)	0

Todos sueltan el lock en `onDisable`, así que un cambio de escena no deja locks colgados. El hit-stop también pone `timeScale` en 0, pero no toma lock.

Lectores de `IsUiPaused`: `HeatManager.cs:290`, `PlayerMovement.cs:314` y `WeaponPresentationController.cs:448`.

`CanPause` (`PauseMenuUI.cs:163-182`) no abre la pausa sobre level-up o crafteo, ni con la run terminada. Escape cierra primero el popup de más arriba (panel de settings, tab DEV) y recién después reanuda.

Salidas desde gameplay

Acción	Código	Ruta
Pausa → Título	<code>PauseMenuUI.cs:232-246</code>	<code>SceneNavigation.LoadTitle()</code> directo. Si falla, vuelve a pausar
Fin de run → Menú	<code>RunEndScreenUI.cs:145-148</code>	<code>SceneNavigation.LoadTitle()</code> directo
Fin de run → Retry	<code>RunEndScreenUI.cs:134-143</code>	<code>GameManager.ResetTimeScaleForReload()</code> y <code>SceneManager.LoadScene(buildIndex activo)</code>

Deuda / puntos abiertos

- Las salidas desde gameplay no usan `ScrapSceneTransition`, aunque la instancia sigue viva en `DontDestroyOnLoad` si el juego arrancó en el título. Título → juego tiene cortina. Juego → título y Retry cortan en seco.
- Retry salta `SceneNavigation` (`RunEndScreenUI.cs:142`). Queda un segundo punto de carga que resetea `timeScale` por su cuenta y que, si se agrega lógica a `SceneNavigation.Load` (transición, limpieza), no la va a heredar.
- `test_balance.unity` está habilitada en el build sin destino ni botón. Sandbox y Enemies Testing también se empaquetan aunque sus botones estén ocultos. Hoy todo build de release las incluye.
- `GameplayPause.Reset()` (`GameplayPause.cs:44-47`) no tiene llamadores.
- Arrancar `GameplayScene` desde el editor no crea `UserSettingsService`, así que la sensibilidad y los volúmenes quedan en los valores de escena. No hay bootstrap que lo cubra.

10 Settings y accesibilidad

Hay **dos almacenes de preferencias**, que se solapan:

- 1. **UserSettingsService**. Controles, audio y feedback básico. Lo guarda en `PlayerPrefs` y `UserSettingsApplier` lo empuja a los componentes vivos.
- 2. **PresentationAccessibilityRuntime**. Reduced motion, reduced shake y reduced flash, más el texto de combate. Es un estado estático. Lo persiste `SaveManager` dentro del save de meta-progresión (JSON).

Reduced motion existe en los dos, y el título y la pausa escriben en almacenes distintos.

Scripts

Script	Rol
Assets/Scripts/Settings/UserSettingsData.cs	DTO serializable, defaults, rangos, flags <code>UserSettingsChange</code>
Assets/Scripts/Settings/UserSettingsService.cs	Singleton persistente: carga, set, reset, guardado con debounce
Assets/Scripts/Settings/UserSettingsApplier.cs	Aplica a cámara, audio, presentación de armas, feedback de combate y reacciones de enemigos
Assets/Scripts/Settings/Accessibility/PresentationAccessibilitySettings.cs	DTO y snapshot inmutable <code>PresentationAccessibilityState</code>
Assets/Scripts/Settings/Accessibility/PresentationAccessibilityRuntime.cs	Estado global y evento <code>Changed</code>
Assets/Scripts/Meta/SaveManager.cs	Persiste la accesibilidad en <code>scrapwaves_save.json</code>
Assets/Scripts/UI/MainMenu/SettingsScreenUI.cs	Pantalla de settings del título
Assets/Scripts/Weapon/UI/PauseMenuUI.cs	Settings dentro de la pausa

Opciones

Opción	Almacén	Default	Rango	Título	Pausa
Sensibilidad horizontal	UserSettings	0,12	0,02–0,4	Sí	Sí
Sensibilidad vertical	UserSettings	0,12	0,02–0,4	Sí	Sí
Invertir Y	UserSettings	No	—	Sí	Sí
Volumen SFX	UserSettings	0,2	0–1	Sí	Sí
Volumen música	UserSettings	0,2	0–1	Sí	Sí
Reduced motion	Ambos	No	—	Sí (UserSettings)	Sí (escribe en los dos)
Screen shake	UserSettings	Sí	—	Sí	No
Screen flash	UserSettings	Sí	—	Sí	No
Reduced shake	Accesibilidad	No	—	No	Sí
Reduced flash	Accesibilidad	No	—	No	Sí
Texto de combate	Accesibilidad	Full	Off / ImportantOnly / Full	No	Off o Full. ImportantOnly no se puede elegir (<code>PauseMenuUI.cs:548</code>)

Opción	Almacén	Default	Rango	Título	Pausa
Escala del texto de combate	Accesibilidad	1	0,75–1,25	No	Sí

Defaults y rangos en `UserSettingsData.cs:7-19` y `PresentationAccessibilitySettings.cs:18-27`. El título tiene reset por categoría (Controles, Audio, Feedback) y reset total (`UserSettingsService.cs:210-228`). La pausa no tiene reset.

Persistencia

UserSettings. Clave `ScrapWaves.UserSettings.v1.Data` de `PlayerPrefs`, con JSON vía `JsonUtility` (`UserSettingsService.cs:8,254-284,351-366`).

1. **Carga perezosa.** La primera lectura de cualquier propiedad dispara `EnsureInitialized`. Si falla la lectura o el JSON, usa los defaults con un warning único.
2. **Migración.** `MigrateLegacyAudioDefaults` (:286-302) reemplaza SFX = 1,0 y música = 0,45 (los defaults viejos) por 0,2 y guarda.
3. **Sanitize.** Clampa los rangos y cambia NaN o infinito por el default (`UserSettingsData.cs:47-88`).
4. **Set.** Cada setter sanitiza, compara, emite `Changed(flag)` y agenda el guardado a 0,2 s en unscaled time (:333-349). `FlushPendingSave` se fuerza en pausa de app, al salir, en `OnDestroy`, al cerrar el panel de settings de la pausa y al reanudar.

Accesibilidad. `SaveManager` (bootstrap automático `BeforeSceneLoad`, `SaveManager.cs:34-43`) carga `Application.persistentDataPath/scrapwaves_save.json` y llama `PresentationAccessibilityRuntime.Apply`. `SetPresentationAccessibility` (:229-249) sanitiza, aplica y guarda solo si cambió. `ResetProgress` conserva la accesibilidad (:210-222).

`PresentationAccessibilityRuntime.Apply` (`PresentationAccessibilityRuntime.cs:28-42`) emite `Changed` solo si el snapshot cambió. El estado se resetea en `SubsystemRegistration`.

Aplicación

`UserSettingsApplier` vive junto al servicio en `PersistentSystemsRoot` del título (`TitleScreen.unity:11293-11317`) y viaja con él por `DontDestroyOnLoad`.

1. Se suscribe a `BecameAvailable` de `ThirdPersonCamera`, `AudioManager`, `WeaponPresentationController` y `PlayerCombatFeedback`, y a `sceneLoaded` (`UserSettingsApplier.cs:17-28`).
2. En cada `sceneLoaded` redescubre targets con `FindObjectsByType` y aplica todo (:99-110).
3. En `Changed` aplica solo el grupo afectado (:80-97).

Target	Qué recibe	Código
<code>ThirdPersonCamera</code>	Sensibilidades, Invert Y, <code>ScreenShakeEnabled = shake && !reducedMotion</code>	:234-240
<code>AudioManager</code>	<code>SfxVolume</code> , <code>MusicVolume</code>	:242-246
<code>WeaponPresentationController</code>	Reduced motion, shake, flash	:248-254
<code>PlayerCombatFeedback</code>	Reduced motion, flash (apaga el flash de daño)	:256-261
<code>EnemyReactionRuntime</code>	Reduced motion, flash	:227-232

Del lado de accesibilidad, `WeaponPresentationController` se suscribe a `PresentationAccessibilityRuntime.Changed` y copia los cinco campos a sus `GameFeelRuntimeOptions` (`WeaponPresentationController.cs:645-654`). `CombatTextDirector` lee `Current` para modo y escala. Reduced shake multiplica los impulsos de cámara por 0,25 (`CameraFeedbackController.cs:11`).

Otros lectores directos de `UserSettingsService.Instance`:

- `ScrapSceneTransition`: reduced motion y volumen SFX.
- `MenuAudioFeedback`: volumen SFX.
- `TitleScreenController`: preferencias de presentación del menú.

Pausa y reduced motion

- El toggle de la pausa escribe en los dos almacenes (`PauseMenuUI.cs:539-544`).
- Al abrir la pausa, `SyncReducedMotionToPresentationRuntime` copia el valor de `UserSettings` a `Accesibilidad` si difieren (`:334-344`).
- El título no hace esa copia.

Deuda / puntos abiertos

- **Un volumen SFX al 100 % se pierde al reiniciar.** La migración legacy compara contra 1,0 y 0,45 en cada carga, sin versión ni marca de "ya migrado" (`UserSettingsService.cs:286-302`). Un usuario que sube el SFX al máximo, o deja la música en 45 %, vuelve a 20 % en el próximo arranque.
- Hay dos fuentes de verdad para reduced motion. Si se cambia en el título, `PresentationAccessibilityRuntime` queda desfasado hasta que se abre la pausa. En `WeaponPresentationController` gana el último que escribió (`WeaponPresentationController.cs:418-430` contra `:645-654`).
- Shake y flash están partidos: "Screen shake / flash" (on/off, solo en el título) y "Reduced shake / flash" (atenuar, solo en la pausa). Son cuatro toggles en dos pantallas distintas para dos conceptos parecidos.
- El título no expone texto de combate ni reduced shake/flash. La pausa no expone screen shake/flash ni reset.
- `CombatTextMode.ImportantOnly` existe y está soportado por `CombatTextVisibilityPolicy`, pero la UI lo mapea a Off/Full (`PauseMenuUI.cs:548`).
- `UserSettingsService` solo existe si la sesión pasó por `TitleScreen`. `SaveManager` sí tiene bootstrap. Al arrancar `GameplayScene` directo, la accesibilidad se aplica y los settings no.
- Con reduced motion, el applier apaga todo el shake de cámara (`UserSettingsApplier.cs:239`). Los factores de reduced motion de `ThirdPersonCamera` (0,2 / 0,35 / 0) quedan casi sin uso. Ver `11-Camara-y-aim.md`.

11 Dormancia, pools y rendimiento

La idea es que una run larga no se llene de GameObjects ni haga Instantiate/Destroy por cada enemigo. Hay tres capas. Pools por prefab para enemigos, proyectiles, áreas y VFX. Un techo global de activos. Y la dormancia por altura, que congela al enemigo que quedó en un piso de abajo en vez de borrarlo. Lo que sigue describe el código y los valores de `GameplayScene` al 23-09.

Scripts

Script	Rol
<code>EnemyPoolRegistry</code>	Singleton, un <code>EnemyPrefabPool</code> por prefab. También crea los pools de FX si faltan
<code>EnemyPrefabPool</code>	Cola de inactivos de un prefab, prewarm, crecimiento hasta <code>MaxSize</code>
<code>SwarmPooledEnemy</code>	Marca de pertenencia. Resetea al sacar/devolver y enruta <code>Despawn()</code>
<code>IEnemySpawnLifecycle</code>	Hook <code>OnPoolSpawn</code> / <code>OnPoolDespawn</code> (<code>IEnemySpawnLifecycle.cs:4-8</code>)
<code>EnemyLifecycleCoordinator</code>	Limpieza de fin de Overheat (solo elites) y "Clear all" de QA
<code>EnemyVerticalEngagement</code>	Desenganche y dormancia por altura
<code>EnemyDormancyRegistry</code>	Lista de dormidos y techo con expulsión
<code>EnemyRegistry</code> / <code>EnemyRegistryMember</code>	Lista estática de activos. Targeting, separación y cap
<code>EnemyMovementSteering</code>	Punto de persecución en anillo, weave y separación muestreada
<code>EnemyProjectilePool</code> , <code>EnemyTimedAreaPool</code> , <code>ExplosionRadiusVfxPool</code>	Pools de apoyo
<code>EnemyPoolProfiler</code> / <code>EnemyPoolProfilerHud</code>	Contadores de QA y HUD IMGUI
<code>SwarmSpawner</code> + <code>SwarmEnemyPool</code>	Legacy, [Obsolete] (<code>SwarmSpawner.cs:3</code>)

Pool de enemigos

En la escena, el registry vive en `GameplayPools` (`GameplayScene.unity:29250-29254`) con `_useEnemyPool: 1`, `_entries: []`, `_allowPoolGrowth: 1` y la ruleta `DefaultEnemySpawnRoulette`. Corre con `DefaultExecutionOrder(-42)` (`EnemyPoolRegistry.cs:10`).

Origen del pool	Prewarm	Máximo	Dónde
Entrada manual (<code>Entry</code>)	16	128	<code>EnemyPoolRegistry.cs:17-18</code> (en escena no hay)
Prefab de la ruleta	<code>max(8, BatchSize × 2)</code>	256	<code>EnemyPoolRegistry.cs:115-116</code>
Prefab no registrado, primer <code>TryGet</code>	8	128	<code>EnemyPoolRegistry.cs:173</code>

La ruleta tiene seis prefabs: `EnemyPro`, `Drone`, `Chaser` y sus tres variantes. Los batch son 4/4/3/2/2/2, así que todos quedan en el piso de 8. Al cargar se instancian 48 enemigos inactivos. Los elites de la oleada entran por el autoregistro (8/128). `BossManager` no pasa por el pool: hace `Instantiate` (`BossManager.cs:189`).

`EnemyPrefabPool.TryGet` saca de la cola. Si está vacía y hay crecimiento, crea hasta `_maxSize`. Si no, devuelve `null` (`EnemyPrefabPool.cs:42-64`). El prewarm se recorta a `maxSize (:30)`.

Ciclo de vida de un enemigo orbital

1. `OrbitalSpawner.SpawnRouletteWave` corta el batch si `EnemyRegistry.ActiveCount >= MaxActiveEnemies` (`OrbitalSpawner.cs:231`).
2. `OrbitalSpawnPlacement` pide `EnemyPoolRegistry.TryGet`. Si falla, hace `Instantiate` sin pool (`OrbitalSpawnPlacement.cs:118-126`).
3. `EnemyPrefabPool.ActivateInstance: SetActive(true)` (entra a `EnemyRegistry` por `EnemyRegistryMember.OnEnable`), `_leasedCount++` y `NotifySpawned()` (`EnemyPrefabPool.cs:119-124`).
4. `SwarmPooledEnemy.ResetForPoolSpawn: EnemyHealth.PrepareForPoolSpawn`, `PrepareForSpawn` de los followers, agrega los drops meta si faltan, `EnemyVerticalEngagement.EnsureOn + ResetEngagement`, y llama `OnPoolSpawn` en cada `IEnemySpawnLifecycle` (`SwarmPooledEnemy.cs:59-75`).
5. Se posiciona sobre el suelo. Si no encuentra pie válido, vuelve al pool o se destruye (`OrbitalSpawnPlacement.cs:158-163`).
6. Vive. Puede desengancharse y dormirse (ver abajo).
7. Muerte: `EnemyHealth.CompleteDeath` dispara `OnDied`, stats y `FinalizeDeath`. Si está bound, `Despawn()`; si no, `Destroy` (`EnemyHealth.cs:165-185`).
8. `Despawn()` va al pool legacy, al registry o a `Destroy` (`SwarmPooledEnemy.cs:41-57`).
9. `EnemyPrefabPool.Release: NotifyDespawned()` (followers `OnDespawned + OnPoolDespawn`, que despierta a un dormido), después `SetActive(false)` (sale de `EnemyRegistry`), `reparent`, `_leasedCount--` y vuelta a la cola (`EnemyPrefabPool.cs:66-81`).

Implementan `IEnemySpawnLifecycle: EnemyBehaviorBase`, `EliteObjectiveMarker` y `EnemyVerticalEngagement`. El orden importa. `OnPoolDespawn` corre antes del `SetActive(false)` porque desactivar no restaura los `.enabled` que tocó la dormancia (`EnemyVerticalEngagement.cs:396-399`).

`EnemyLifecycleCoordinator.OnOverheatEnded` solo limpia elites de la oleada; el swarm orbital queda (`EnemyLifecycleCoordinator.cs:13-22`). `ClearAllForQa` libera todos los pools, el legacy, el orbital, las zonas y los elites, y resetea los contadores (:25-56).

Techo de activos

`MaxActiveEnemies` sale del perfil si hay uno y, si no, del fallback serializado (`OrbitalSpawner.cs:101`).

`BalanceTuningHub` inyecta `SpawnBalanceProfile.asset`, que tiene `_maxActiveEnemies: 300` (línea 113 del asset). El fallback de escena también es 300 (`GameplayScene.unity:27720`). El recorte es por enemigo dentro del batch: el loop hace `break` apenas se llega al techo.

`EnemyRegistry.ActiveCount` cuenta todo lo registrado, **dormidos incluidos** (`EnemyRegistry.cs:10`, comentario en :59-61). El profiler separa `AwakeActiveCount = ActiveCount - DormantCount` (`EnemyPoolProfiler.cs:19`).

Vertical engagement y dormancia

Parámetro	Valor	Dónde
<code>_belowThreshold</code>	40 u (<code>player.y - enemy.y</code>)	<code>EnemyVerticalEngagement.cs:29</code>
<code>_dormancyAfterSeconds</code>	10 s	:32
Sonda de suelo al dormir	+2 u hacia arriba, 60 u hacia abajo, capas <code>Terrain+Default</code>	:14-15, :389-394
Techo de dormidos	150	<code>EnemyDormancyRegistry.cs:16</code>

Ningún prefab ni la escena serializan el componente: se agrega en runtime con `EnsureOn`, así que rigen los defaults.

1. **Desenganchado.** Si el enemigo queda más de 40 u por debajo del jugador, su id entra a `s_DisengagedIds` (:169-179). `EnemyRegistry` lo saltea para targeting, succión y separación (`EnemyRegistry.cs:62`, :440). Si está

arriba o en el mismo piso, sigue enganchado y puede caer hacia el jugador.

2. **Dormido.** Después de 10 s seguidos abajo, `EnterDormancy` (:213-276): destruye el `WeaponMovementFreezeStatus`, apaga behaviors antes que followers, pone el rigidbody kinematic, lo apoya en el piso, limpia knockback e invencibilidad, y apaga `CharacterController`, colliders, renderers y `EnemyHitFeedback`. Queda en el mundo y se registra en `EnemyDormancyRegistry`.
3. **Despertar.** Cuando el jugador baja a menos de 40 u, `ExitDormancy` restaura el estado cacheado. Si tiene behavior, reactiva solo el behavior y deja que su `OnEnable` decida los followers (:279-316).
4. **Techo.** Con más de 150 dormidos, expulsa primero los de prefabs cuyo pool no tiene stock y después los más lejanos. Los pooled vuelven con `Despawn()`; el resto se destruye (`EnemyDormancyRegistry.cs:75-124`).

Exenciones: cualquier `GigaWormBehavior` (`EnsureOn` devuelve null, `Awake` fuerza exento; :92-106). Los elites de la oleada se marcan después de obtener la instancia (`OverheatEliteWaveSpawner.cs:317-321`). Los bosses, en `BossManager.cs:245-251`. `ResetEngagement` devuelve el valor de fábrica al reciclar, así un elite reciclado no queda exento para siempre (:72-79, :148-154).

Steering

`EnemyMovementSteering` no asigna slots únicos. Cada enemigo sortea un ángulo de slot y una fase de weave (`EnemyMovementSteering.cs:222-226`). Persigue un punto en un anillo alrededor del jugador, de radio `orbitRadius`, que se achica cerca para que el melee cierre el contacto (:194-201). La separación usa un caché de posiciones que se refresca una vez por frame desde `EnemyRegistry.CollectActive`, sin desenganchados (:114-138). Toma `maxSeparationSamples` vecinos con stride, con costo $O(\text{muestras})$ y no $O(n)$ (:141-178). El perfil `MeleeSwarm` usa orbit 1.8, separación 0.7 con radio 1.25 y 8 muestras (:18-26).

Pools de apoyo

`EnemyPoolRegistry.EnsureExists` crea los tres si faltan (`EnemyPoolRegistry.cs:77-79`). En escena ya están sobre `GameplayPools`.

Pool	Usuarios	Escena	Tope efectivo
<code>EnemyProjectilePool</code>	<code>FlyingRangedBehavior</code> , <code>BomberDroneBehavior</code>	prefab asignado, 32 iniciales, máximo 256 por prefab (:29267-29271)	Sí, cuenta el total por prefab (<code>EnemyProjectilePool.cs:158-161</code>)
<code>EnemyTimedAreaPool</code>	<code>FireArea</code> (Hellfire), <code>EnemyC4</code> (Bomber), <code>CorrosiveSlimeArea</code>	4 / 32 por prefab (:29284-29285)	No. Ver deuda
<code>ExplosionRadiusVfxPool</code>	<code>ExplosionRadiusVfx.Spawn</code>	8 / 32 (:29298-29299)	No. Ver deuda

Los tres, si no hay instancia o se llega al tope, caen a `Instantiate` y el objeto se destruye al terminar. El comentario "Pooling = TODO futuro" de `EnemyProjectile.cs:9` ya no describe el runtime: `Consume()` devuelve al pool (:109-110).

Profiler y HUD

`EnemyPoolProfiler` cuenta `Instantiate`, `Destroy`, `Get` y `Release` del pool, más `RegistryActiveCount`, `DormantCount`, `AwakeActiveCount` e `InactiveEnemyObjects`. Este último se calcula con `FindObjectsByType<EnemyHealth>` (`EnemyPoolProfiler.cs:8-46`). Los contadores de alloc los alimentan también los pools de proyectiles, áreas y VFX: no son solo de enemigos.

`EnemyPoolProfilerHud` está en `GameplayPools` con `_visible: 0` (`GameplayScene.unity:29312`). Solo dibuja si `_visible` y `QaPanels.Active == Qa`, o sea con el panel F1 (`EnemyPoolProfilerHud.cs:28`). No tiene hotkey propio ni setter, así que en `GameplayScene` nunca se ve. Tampoco muestra los dormidos. Ese dato está en el menú F3,

QaCoreLoopMenu.cs:182.

Legacy

SwarmSpawner está en SpawnPoint con `m_Enabled: 0` (GameplayScene.unity:25409). Su `_maxActiveEnemies: 200` no aplica. En cambio, SwarmEnemyPool está activo y habilitado (`:34234`) con `Enemy.prefab`, 64 iniciales y máximo 512, y su `Awake` hace el prewarm completo (SwarmEnemyPool.cs:83-87).

Tests

- `Assets/Tests/Editor/EnemyDormancyTests.cs`: dormancia que deja el objeto inerte y restaura el estado exacto, `OnPoolDespawn` que despierta, exento que nunca duerme, `ResetEngagement` que limpia el exento de un elite reciclado, techo que expulsa al más lejano, y `EliteObjectiveMarker` / contador de elites cuando uno sale sin morir.
- `Assets/Tests/Editor/EnemyPoolLifecycleTests.cs`: `PrepareForPoolSpawn` limpia invencibilidad y rellena vida. `Despawn` enruta al registry cuando está bound.
- Sin cobertura: la prioridad por pool sin stock en la expulsión, los topes de `EnemyPrefabPool` y los pools de FX.

Deuda / puntos abiertos

- **SwarmEnemyPool legacy instancia 64 Enemy.prefab al cargar la escena** aunque SwarmSpawner está deshabilitado. Son objetos muertos en memoria. Sacar el componente o bajarle `_initialPoolSize`.
- **El tope de EnemyTimedAreaPool no funciona.** `CountForPrefab` devuelve `queue.Count`, que es solo lo inactivo (`EnemyTimedAreaPool.cs:108-112`). Con la cola vacía siempre da `0 < 32` y crea otra. Pasa lo mismo en `ExplosionRadiusVfxPool.TotalCount()` => `_inactive.Count (:87)`. El crecimiento no tiene techo.
- **Con el pool agotado, el orbital spawna sin pool.** Los 256 por prefab caen a `Instantiate` (`OrbitalSpawnPlacement.cs:123-126`). Esas instancias no pasan por `NotifySpawned`, así que no reciben `EnemyVerticalEngagement` ni los drops meta, salvo que el prefab ya los traiga. Nunca se duermen y mueren con `Destroy`. El comentario de `EnemyDormancyRegistry.cs:10-12` dice que el tipo sin stock "deja de spawnear", y no es así.
- **Los dormidos cuentan para el techo.** 150 dormidos sobre 300 de `MaxActiveEnemies` dejan como mínimo 150 despiertos en el piso del jugador. Hay que decidir si el techo debería mirar `AwakeActiveCount`.
- `EnemyPoolRegistry.Release` **no hace nada si el prefab no está registrado** (`:191`). La instancia queda activa y nadie se entera.
- **El HUD del profiler es inalcanzable en GameplayScene** (`_visible: 0`, sin toggle) y no muestra dormidos. Los contadores de `Instantiate/Destroy` mezclan enemigos con proyectiles, áreas y VFX.
- **Comentarios viejos.** El summary de `EnemyVerticalEngagement.cs:4-8` todavía dice que el enemigo "vuelve al pool" tras el sleep; hoy queda congelado. `EnemyProjectile.cs:9` dice "Pooling = TODO".
- **BossManager usa Instantiate sin pool** (`BossManager.cs:189`) y no llama a `EnemyPoolProfiler.RegisterInstantiate`, así que los bosses no aparecen en los contadores. Es poco volumen.

12 Herramientas de editor y tests

Inventario de lo que hay en `main` para construir assets, validar y probar sin jugar una run entera. Ninguna de estas piezas es autoridad de gameplay: generan prefabs y escenas, fuerzan valores o leen estado. Complementa `06-QA-y-debug.md` con rutas de menú, conteo de tests y el mapa real de teclas.

Scripts

Carpeta	Contenido
<code>Assets/Scripts/Editor/ (+ MainMenu/)</code>	Builders de animación del jugador, wearables, rear threat, UI de run, escena de juego, título
<code>Assets/Scripts/Balance/Editor/</code>	<code>BalanceTuningHubEditor</code> (inspector en vivo del hub)
<code>Assets/Scripts/Economy/Editor/</code>	Import de CSV de balance, setup de economía, <code>BalanceAutoImporter</code>
<code>Assets/Scripts/Enemy/Editor/</code>	<code>DestroyerPrefabMenu</code>
<code>Assets/Scripts/Level/Editor/</code>	Compactor, salida, flecha guía, niebla, modelos de la escena
<code>Assets/Scripts/Spawning/Editor/</code>	Asset de ruleta por defecto
<code>Assets/Scripts/Testing/ + Editor/</code>	<code>BalanceTestingController</code> y su builder de escena
<code>Assets/Scripts/Weapon/Editor/ , Weapon/Testing/Editor/</code>	Builders de game feel por arma, HUD, sandbox, snapshot de balance, inspectores
<code>Assets/Tests/Editor/</code>	Suite NUnit, 82 archivos

Hooks automáticos (`[InitializeOnLoad]`): `BalanceAutoImporter` (`Economy/Editor/BalanceAutoImporter.cs:5`) importa los CSV solo si falta `MaterialUsageBalance.asset (:25)`; `PlayerAnimationValidation` (`Editor/PlayerAnimationValidation.cs:9`) y `RearThreatValidation` (`Editor/RearThreatValidation.cs:15`) solo re-registran el callback del Test Runner si quedó una corrida pendiente en `SessionState`.

Inspectores custom: `BalanceTuningHubEditor`, `DebugMonitorEditor`, `WeaponDataEditor`, `WeaponPresentationProfileEditor`. `ReadmeEditor` es el de la plantilla de Unity (`TutorialInfo`).

Menús de editor

54 `[MenuItem]` en `Assets`, todos del proyecto. No hay menús de terceros (`TextMesh Pro` y `TutorialInfo` no aportan). Conviven cuatro raíces: `ScrapWaves/`, `Tools/ScrapWaves/`, `Tools/Scrap Waves/` (con espacio) y `Tools/Scenes/`.

Ruta	Archivo:línea	Qué hace
<code>ScrapWaves/Balance/Import All CSV</code>	<code>Economy/Editor/BalanceImportMenu.cs:13</code>	CSV de <code>Assets/Data/Balance</code> → <code>MaterialUsageBalance + WeaponSO</code>
<code>ScrapWaves/Balance/Create Default Drop Configs</code>	<code>BalanceImportMenu.cs:30</code>	Crea configs de drop por defecto
<code>ScrapWaves/Economy/Apply Drop Art Updates (CellBattery + PlasticV2)</code>	<code>Economy/Editor/DropAssetsUpdateMenu.cs:10</code>	Cambia modelos de dos pickups

Ruta	Archivo:línea	Qué hace
ScrapWaves/Economy/Add Economy To Player Prefab	Economy/Editor/EconomySceneSetupMenu.cs:7	Agrega componentes de economía a <code>player.prefab</code>
ScrapWaves/Economy/Create Crafting Station In Scene	EconomySceneSetupMenu.cs:60	Estación de crafeo en la escena abierta
ScrapWaves/Economy/Wire Material Drop Art Models	EconomySceneSetupMenu.cs:74	Llena <code>MaterialDropVisualCatalog</code>
ScrapWaves/Economy/Create Material Orb Prefab	EconomySceneSetupMenu.cs:138	Prefab de orbe de material
ScrapWaves/Economy/Create Material Pool In Scene	EconomySceneSetupMenu.cs:181	Pool de materiales en la escena
Tools/ScrapWaves/Fix Enemy Skinned Animations (Chaser+Drone)	Editor/EnemySkinnedAnimBuilder.cs:42	Rehace controllers skinned de enemigos
Tools/ScrapWaves/Build Chaser Walk Animation	EnemySkinnedAnimBuilder.cs:55	Solo chaser
Tools/ScrapWaves/Build Slime Hop Animation	EnemySkinnedAnimBuilder.cs:65	Solo slime
Tools/ScrapWaves/Build Drone Fly Animation	EnemySkinnedAnimBuilder.cs:75	Solo drone
Tools/Scenes/Create Gameplay Scene (Level 1)	Editor/GameplaySceneBuilder.cs:15	Copia <code>SampleScene</code> si falta <code>GameplayScene</code> , cablea pools y Build Settings
ScrapWaves/UI/Organize Current Scene Under UI	Editor/GameplayUiSceneMigration.cs:18	Reordena la UI de la escena abierta
ScrapWaves/UI/Migrate All Player Scenes Under UI	GameplayUiSceneMigration.cs:26	Lo mismo en todas las escenas con jugador
Tools/Scrap Waves/Validate Authored Title Screen	Editor/MainMenu/TitleScreenAuthoringValidator.cs:31	Valida la escena de título
Tools/ScrapWaves/Build Placeholder Player Animation	Editor/PlaceholderPlayerAnimationBuilder.cs:29	Clips, controller y attachments del jugador
Tools/ScrapWaves/Refresh Player Animation Controller	PlaceholderPlayerAnimationBuilder.cs:113	Solo clips y controller
Tools/ScrapWaves/Refit Animated Wearable Sockets	PlaceholderPlayerAnimationBuilder.cs:149	Reajusta sockets de wearables
Tools/ScrapWaves/Render Placeholder Player Animation	PlaceholderPlayerAnimationBuilder.cs:322	Renders de preview a <code>.utmp/player-animation</code>
Tools/ScrapWaves/Validate Placeholder Player Animation	Editor/PlayerAnimationValidation.cs:39	Corre 13 fixtures EditMode y guarda XML + resumen

Ruta	Archivo:línea	Qué hace
ScrapWaves/UI/Create Missing Rear Threat Indicator Assets	Editor/RearThreatIndicatorAuthoring.cs:35	Assets del indicador
ScrapWaves/UI/Add Rear Threat Indicator To Current Scene	RearThreatIndicatorAuthoring.cs:78	Lo agrega a la escena abierta
ScrapWaves/UI/Refresh Rear Threat Prefab Preview Meshes	RearThreatIndicatorAuthoring.cs:81	Regenera meshes de preview
ScrapWaves/UI/Add Rear Threat Indicator To All Player Scenes	RearThreatIndicatorAuthoring.cs:203	Migración masiva
ScrapWaves/Validation/Rear Threat/Compile and Render	Editor/RearThreatValidation.cs:63	Compila player y renderiza a Library/CodexValidation/RearThreat
ScrapWaves/Validation/Rear Threat/Render PC and Mobile Previews	RearThreatValidation.cs:88	Solo previews
ScrapWaves/UI/Create Missing Run Menu Assets	Editor/RunMenuPrefabBuilder.cs:35	Assets de menús de run
Tools/ScrapWaves/Audit Animated Wearable Clearance	Editor/WearableAnimationClearanceAudit.cs:31	Hornea la piel en una preview scene y mide choques con wearables
Tools/ScrapWaves/Build Wearable Weapon Mounts	Editor/WearableWeaponMountBuilder.cs:62	Cinco fire points y catálogo del jugador
Tools/ScrapWaves/Render Wearable Weapon Mount Preview	WearableWeaponMountBuilder.cs:160	Preview de montajes
ScrapWaves/Enemies/Create Destroyer Missile Prefab	Enemy/Editor/DestroyerPrefabMenu.cs:17	Prefab de misil
ScrapWaves/Enemies/Create Destroyer Prefab	DestroyerPrefabMenu.cs:49	Destroyer_Boss.prefab con Mouth/WeakPoint placeholder
ScrapWaves/Enemies/Assign Destroyer As Second Boss In Scene	DestroyerPrefabMenu.cs:105	Lo pone en el BossManager de la escena
ScrapWaves/Level/Create Below Level Fog Prefab	Level/Editor/BelowLevelFogPrefabBuilder.cs:16	Plano de niebla
Tools/ScrapWaves/Open Compactor Escape Test	Level/Editor/CompactorEscapeSetupEditor.cs:20	Abre WeaponTestingSandbox y enfoca la puerta
Tools/ScrapWaves/Setup Compactor Escape Test	CompactorEscapeSetupEditor.cs:33	Importa modelo, crea prefab y lo mete en los dos sandboxes
Tools/ScrapWaves/Replace Gameplay Placeholder Models	Level/Editor/GameplayModelsSetupEditor.cs:15	Reemplaza puerta y workbench en GameplayScene

Ruta	Archivo:línea	Qué hace
ScrapWaves/Level/Create Guide Arrow In Scene	Level/Editor/GuideArrowSetupMenu.cs:14	Flecha guía
ScrapWaves/Level/Setup Exit System	Level/Editor/LevelExitSetupEditor.cs:17	Sistema de salida, bosses y HUD V2
ScrapWaves/Spawning/Create Default Enemy Spawn Roulette	Spawning/Editor/EnemySpawnRouletteAssetUtility.cs:9	DefaultEnemySpawnRoulette.asset
Tools/ScrapWaves/Build Balance Testing Scene (test_balance)	Testing/Editor/BalanceTestingSceneBuilder.cs:25	Genera la escena de balance (ver abajo)
Tools/ScrapWaves/Open Balance Testing Scene (test_balance)	BalanceTestingSceneBuilder.cs:202	La abre
Tools/ScrapWaves/Game Feel/Rebuild Combat Text Assets	Weapon/Editor/CombatTextAssetBuilder.cs:20	Assets de texto de combate
Tools/ScrapWaves/Game Feel/Rebuild Flamethrower Production Assets	Weapon/Editor/FlamethrowerAssetBuilder.cs:34	Presentación del lanzallamas
Tools/ScrapWaves/Game Feel/Rebuild Cannon Production Assets	Weapon/Editor/GameFeelFoundationAssetBuilder.cs:38	Presentación del cañón
Tools/ScrapWaves/Game Feel/Rebuild Mortar Production Assets	Weapon/Editor/MortarAssetBuilder.cs:35	Mortero
Tools/ScrapWaves/Game Feel/Rebuild Rocket Launcher Production Assets	Weapon/Editor/RocketLauncherAssetBuilder.cs:35	Lanzacohetes
Tools/ScrapWaves/Game Feel/Rebuild Rotating Blade Production Assets	Weapon/Editor/RotatingBladeAssetBuilder.cs:35	Sierra rotante
ScrapWaves/UI/Build GameplayHud Prefab	Weapon/Editor/GameplayHudPrefabBuilder.cs:21	Prefab de HUD
ScrapWaves/UI/Rebuild BottomStrip In Prefab	GameplayHudPrefabBuilder.cs:49	Franja inferior del HUD
ScrapWaves/UI/Wire SampleScene GameplayHud	GameplayHudPrefabBuilder.cs:259	Cablea el HUD en SampleScene (legacy)
Tools/ScrapWaves/Apply Weapon Balance Sheet Snapshot	Weapon/Editor/WeaponBalanceSheetApplier.cs:25	Pisa WeaponData con valores hardcodeados
Tools/ScrapWaves/Build Weapon Testing Sandbox	Weapon/Testing/Editor/WeaponTestingSandboxSceneBuilder.cs:19	Escena sandbox de armas y dummy

BalanceTuningHubEditor no es menú: en Play muestra intervalo, intensidad y estado de spawn en vivo, con botones de heat 0/40/80/99 % (Balance/Editor/BalanceTuningHubEditor.cs:79-86) y "Disparar Overheat" (:89).

Escena de balance

BalanceTestingController (Testing/BalanceTestingController.cs:10) es un panel IMGUI para probar 1×1.

1. **Build Balance Testing Scene** crea una escena vacía y le agrega luz, suelo, **HeatManager**, pool de proyectiles, jugador, cámara, **EventSystem**, dificultad, estación de crafeo, puntos de power-up y el controller (**BalanceTestingSceneBuilder.cs:26-46**).
2. Guarda en **Assets/Scenes/Testing/test_balance.unity** y la agrega **habilitada** a **Build Settings** (:48-49).
3. En **Play: F1** muestra u oculta el panel y **F2** alterna el mouse entre **UI** y cámara (**BalanceTestingController.cs:81-85**).
4. Botones: spawn +1/+10 por **EnemySpawnKind**, **Destroyer** y **Stalker**, 999 de todo o +50 por material, reset al arma inicial (**AutomaticCannon**), abrir crafeo, power-ups, limpiar enemigos y curar.

Suite de tests

- Todo vive en **Assets/Tests/Editor/**. No hay ningún **.asmdef** en **Assets**: los tests compilan en **Assembly-CSharp-Editor** y solo aparecen en la pestaña **EditMode** (**com.unity.test-framework 1.6.0**). No hay assembly de **PlayMode**.
- 840 atributos: 598 **[Test]**, 233 **[TestCase]** y 9 **[UnityTest]**. Un **[TestCase]** cuenta como un caso. Los **[UnityTest]** entran a **Play** con **EnterPlayMode** desde **EditMode** (6 archivos).

Área	Archivos (atributos)	Total
Armas: lógica y upgrades	WeaponUpgradeEffect 93, WeaponUpgradeMath 34, AutomaticWeaponMount 26, WearableWeaponFiring 21, SandboxWeaponUpgradeData 21, WeaponDiagnostics 13, AutomaticCannonFireLogic 5, MortarTrajectory 5, WeaponDamageAttribution 5, ManualWeaponFireCooldown 4, WeaponManagerSandboxParity 4, ManualProjectileTargetDamage 3, MortarTerrainFilter 2, ActiveAbilityAmmo 2	238
Armas: presentación / game feel	AutomaticCannonPresentation 27, MortarPresentation 21, FlamethrowerPresentation 11, GameFeelFoundation 10, RocketLauncherPresentation 9, RotatingBladePresentation 9	87
Texto de combate y accesibilidad (GameFeel/CombatText)	CombatTextPresentationCore 16, BurnStatusCombatText 9, PresentationAccessibility 8, DamageApplicationResult 6, DamageFeedbackSequenceCore 6, CombatTextAggregateCore 6, CombatTextSpatialDirector 5, CombatTextViewMotion 5, PresentationAccessibilityConsumer 5, CombatTextCleanStyleAsset 2	68
Jugador: animación, wearables, movimiento y stats	PlayerAnimationIntegration 17, StatDisplayFormat 12, PlayerDirectionalDash 10, StatAttribution 9, PlayerStatConsumer 6, PlayerWearableMountFit 6, PlayerTorsoSkinning 5, PlayerWearableSurfaceFollower 5, PlayerXpFlow 3, PlayerWearableClearance 2, PlayerAnimationSandbox 1, PlayerRunPreview 1	77
Apuntado y retícula	GameplayAim 23, ReticlePresentationLogic 10, ReticleHudAuthoring 8, ReticleAimProvider 2	43
Rear threat	RearThreatSensor 39, RearThreatPresentation 15, RearThreatAuthoring 10, RearThreatPlayModeValidation 1	65
Enemigos	EnemyReaction 36, EnemyDormancy 10, EnemyHealthDot 2, EnemyPoolLifecycle 2	50
Overheat / spawn / dificultad	OverheatCycleSpawnScaling 7, HeatSpawnScaling 6, DifficultyManager 2	15
Economía, meta y pasivos	AdvancedTinkering 26, EconomyBalance 6, PassiveltemInventory 6, PassiveltemTestingController 6, ObjectivesMenuUIPresenter 6, PassiveltemManager 5, SandboxPassiveltemData 4, AchievementToastAuthoring 3, AchievementUilconCatalog 2, PickupGroundFall 1	65
UI, menús y navegación	RunMenuUi 27, PauseMenuUI 19, TitleScreenController 12, SceneNavigation 12, GameplayPause 8, GameplayHudAuthoring 7, SupportUiAuthoring 7, ScrapSceneTransition 7, GameplayUiScene 6, UserSettingsService 5, TitleScreenScreenStack 4, WeaponUilconCatalog 2	116
Nivel	CompactorDoorPresentation 6, CompactorEscapeIntegration 6	12
Audio	BgmTrackSelector 4	4

Sin ningún test que los referencie (búsqueda por nombre de clase en `Assets/Tests`):

- `OverheatManager`: cierre de ciclo y paridad elite/boss. `HeatManager` solo aparece como fixture de escalado de spawn.
- `BossManager`, `DestroyerBehavior`, `GigaWormBehavior`, `HellfireSlimeBehavior`, `BomberDroneBehavior`, `ChargerEnemyBehavior`, `FlyingRangedBehavior` y `BossKeyDrop`.
- `OrbitalSpawner`, `ZoneSpawner` y `ExitSpawnPressure`. `OverheatEliteWaveSpawner` y `EnemySpawnRoulette` solo aparecen de pasada (`EnemyDormancyTests.cs:202`, `PlayerStatConsumerTests.cs:86`).
- Craftero: `CraftingStation` sale en una integración del compactor; de `CraftingUI` y `CraftingMenuView` solo se testea el cableado de UI (`RunMenuUiTests`, `GameplayUiSceneTests`). La transacción de craftero no tiene test.
- `AudioManager` aparece una sola vez (`UserSettingsServiceTests.cs:149`).
- La dormancia sí está cubierta (`EnemyDormancyTests`, 10).

Mapa de teclas de QA

Todo usa el Input System nuevo (`Keyboard.current`). `activeInputHandler: 1` en `ProjectSettings`. La única rama legacy es `Input.GetKeyDown(KeyCode.Q)` en `DebugMonitor.cs:62` bajo `#else`.

Tecla	Script	Archivo:línea	Qué hace
F3 (default)	<code>DebugUI</code>	<code>Weapon/UI/DebugUI.cs:21, :104</code>	Muestra u oculta el overlay
P / Numpad- / Numpad+ (default)	<code>DebugUI</code>	<code>DebugUI.cs:22-24, :113-120</code>	Pausa y escala de tiempo, solo con el overlay visible
F2	<code>QaRuntimeTweaker</code>	<code>Spawning/QaRuntimeTweaker.cs:126</code>	Panel <code>Balance</code>
F3	<code>QaCoreLoopMenu</code>	<code>Spawning/QaCoreLoopMenu.cs:75</code>	Panel <code>CoreLoop</code>
F1	<code>EnemiesTestingHarness</code> → <code>QaPanels</code>	<code>Spawning/EnemiesTestingHarness.cs:149</code>	Panel <code>Qa</code> ; <code>QaPanels</code> (<code>Spawning/QaPanels.cs:22</code>) solo guarda qué panel está activo, sin input propio
Numpad 1 / 4 / 0	<code>EnemiesTestingHarness</code>	<code>EnemiesTestingHarness.cs:152-158</code>	Tirar ruleta, loguear stats y limpiar
Ctrl+Numpad 9	<code>DebugCrafting</code>	<code>Debug/DebugCrafting.cs:65-66</code>	999 de cada material
Numpad 0	<code>DebugInfiniteHealth</code>	<code>Debug/DebugInfiniteHealth.cs:35</code>	Vida infinita on/off
Q	<code>DebugMonitor</code>	<code>Weapon/DebugMonitor.cs:60</code>	Solo loguea el uso de la habilidad
F1 / F2	<code>BalanceTestingController</code>	<code>Testing/BalanceTestingController.cs:81-85</code>	Panel y modo de mouse en <code>test_balance</code>

Qué hay en `Assets/Scenes/GameplayScene.unity` (GUID del `.meta` buscado en el YAML):

Componente	GameObject	<code>m_IsActive / m_Enabled</code>	Nota
<code>DebugUI</code>	<code>DebugUI</code>	1 / 1 (línea 11585)	<code>_visibleOnPlay: 0</code> ; teclas serializadas 96/30/31/19

Componente	GameObject	m_IsActive / m_Enabled	Nota
DebugCrafting	DEBUGTESTINVENTORY	1 / 1 (15554)	999
DebugInfiniteHealth	DEBUG_InfiniteHealth	1 / 1 (27953)	_startEnabled: 0
BalanceTuningHub	BalanceTuning	1 / 1 (17910)	—
DebugMonitor	vía player.prefab	habilitado	Override de escena _logStateChanges: 0
QaRuntimeTweaker, QaCoreLoopMenu, EnemiesTestingHarness	—	ausentes	Solo están en Testing/enemiesTesting.unity, que no tiene DebugUI

F3: hoy no hay conflicto en ninguna escena, porque DebugUI y QaCoreLoopMenu nunca están juntos. Si se agrega QaCoreLoopMenu a GameplayScene, o DebugUI a enemiesTesting, F3 abre los dos a la vez.

Deuda / puntos abiertos

- Con el enum Key de Input System 1.19, los valores serializados de DebugUI en GameplayScene son F3=96 y P=30, pero _timeSlowerKey: 31 es **Q** y _timeFasterKey: 19 es **E**, no Numpad-/+ . Con el overlay abierto, Q (habilidad, WeaponManager.cs:389) baja el time scale y E (interactuar, CraftingStation.cs:69 y ExitDoor.cs:177) lo sube. Además 06-QA-y-debug.md no menciona P ni la escala de tiempo.
- DebugUI, DebugCrafting y DebugInfiniteHealth están activos en GameplayScene y no tienen ninguna guarda isDebugBuild, UNITY_EDITOR ni DEVELOPMENT_BUILD. Un build de release trae F3, Ctrl+Numpad 9 y Numpad 0. El propio summary de DebugInfiniteHealth dice que era para SampleScene y para borrar antes de release (DebugInfiniteHealth.cs:9-10).
- Build Settings tiene habilitadas WeaponTestingSandbox, enemiesTesting y test_balance. Además, el builder de balance vuelve a habilitar test_balance cada vez que se corre (BalanceTestingSceneBuilder.cs:49).
- Hay dos fuentes que escriben WeaponData: Import All CSV (BalanceImportMenu.cs:24) y Apply Weapon Balance Sheet Snapshot, que tiene valores hardcodeados (WeaponBalanceSheetApplier.cs:36+). La última que se corre gana.
- No existe una assembly de PlayMode ni asmdefs de tests. Los 9 [UnityTest] dependen de EnterPlayMode dentro de EditMode, y las validaciones por menú exigen escenas guardadas (PlayerAnimationValidation.cs:72-74).
- El núcleo del loop no tiene tests: OverheatManager, BossManager, spawners orbital, de zona y de oleada, behaviors de variantes y bosses, y la transacción de crafeo.
- Las raíces de menú están partidas (ScrapWaves/, Tools/ScrapWaves/, Tools/Scrap Waves/, Tools/Scenes/). Wire SampleScene GameplayHud todavía apunta a la escena legacy.